EE5311 – Digital IC Design

End-Semester Project Report

6×6 Unsigned Wallace Tree Multiplier

with a Carry-Bypass Vector-Merge Adder

SKY130 | xschem | KLayout | Magic | Netgen | ngspice

Shaun Joseph Alappatt
Roll No.: EE26E007

Indian Institute of Technology Madras
November 2026

Contents

1 Introduction and Specification **3**

1.1 Tools and process . . . 3

1.2 Supplied library cells . . . 3

2 Architecture Overview **3**

3 Basic Building Blocks **4**

3.1 AND2 (`and2`) . . . 4

3.2 True-output full adder (`fa_true`) . . . 4

3.3 Half adder (`ha`) . . . 4

3.4 Naming convention for internal signals . . . 4

4 Partial-Product Generator (`ppgen`) **5**

4.1 Function . . . 5

4.2 Schematic construction . . . 5

4.3 Layout . . . 6

5 Wallace Tree Reduction **6**

5.1 Principle . . . 6

5.2 Common rules followed in every stage . . . 6

5.3 Stage 1: 6 rows $\rightarrow$ 4 rows . . . 7

5.4 Stage 2: 4 rows $\rightarrow$ 3 rows . . . 10

5.5 Stage 3: 3 rows $\rightarrow$ 2 rows . . . 11

5.6 Summary of the tree . . . 13

6 Vector Merge: 3-bit Carry-Bypass Adder **13**

6.1 Why a carry-bypass adder . . . 13

6.2 Choice of 3-bit blocks . . . 13

6.3 The `cba3` block . . . 14

6.4 Complete vector merge (`vectormerge`) . . . 15

7 Top-Level Schematic and Functional Verification **16**

7.1 Top-level schematic . . . 16

7.2 Verification procedure . . . 16

8 Layout **16**

8.1 Design flow . . . 16

8.2 Issues found and fixed during layout . . . 20

9 Physical Verification: DRC and LVS **21**

9.1 Design rule check . . . 21

9.2 Layout versus schematic . . . 21

9.3 Parasitic extraction 22

10 Critical Path Analysis **22**

10.1 Phase 1: textbook (analytical) estimate 23

10.2 Phase 2: checking the prediction in SPICE 24

10.3 Phase 3: simulation-based search 24

10.4 Critical path and activating input 25

11 Critical-Path Delay: Pre-layout vs Post-layout **26**

11.1 Testbench 26

11.2 Results 27

11.3 Simulator output 27

11.4 Waveforms 28

11.5 Discussion 30

11.6 Limitation 30

12 Conclusion **31**

1 Introduction and Specification

The goal of the project is a full-custom 6×6 unsigned multiplier

$$P_{11:0} = A_{5:0} \times B_{5:0}, \qquad 0 \le P \le 63 \times 63 = 3969,$$

built as a Wallace tree. At each level of the tree, full adders (3:2 counters) and half adders (2:2 counters) reduce the number of rows by roughly a factor of 3/2, until only two rows remain. The two rows are then added by a conventional two-operand adder, the *vector merge*.

The design requirements were:

- a schematic and a DRC/LVS-clean layout of the complete multiplier;
- half adders realised as full adders with one input tied to logic 0;
- a ripple-carry vector merge, with a bonus for a faster adder. **This design uses a carry-bypass adder with 3-bit blocks** (the bonus option);
- identification of the critical path and an input vector that activates it;
- pre-layout and post-layout (RC-extracted) simulation of that vector, and a comparison of the critical-path delay.

1.1 Tools and process

Task	Tool / setting
Technology	SkyWater SKY130 (sky130B PDK), $V_{DD} = 1.8\,\mathrm{V}$
Schematic capture, netlisting	xschem
Layout	KLayout (cell library "`LIB`")
Design rule check	KLayout SKY130 DRC deck
Extraction	Magic (LVS netlist and RC-extracted netlist)
Layout vs. schematic	Netgen
Circuit simulation	ngspice, `sky130.lib.spice tt` corner, $27\,^{\circ}\mathrm{C}$

Table 1: Design environment.

1.2 Supplied library cells

The provided library contained layouts (and matching schematics/symbols) of: a full adder producing the *complements* of sum and carry (`fa`), a 2-input XOR (`xor2`), a 4-input NAND (`nand4`), an inverter (`inv`), a 2:1 multiplexer (`mux2`) and a 2-input NAND (`nand2`). All other blocks in this report were built hierarchically from these cells.

2 Architecture Overview

The hierarchy of the design is:

`wallace6x6` $\rightarrow$ {`ppgen`, `stage1`, `stage2`, `stage3`, `vectormerge`} $\rightarrow$ {`cba3`, `fa_true`, `ha`, `and2`} $\rightarrow$ library cells.

Table 2 summarises how the tree reduces the 36 partial-product bits. Every full adder removes one bit (3 in, 2 out); a half adder keeps the count but moves a bit into the next column.

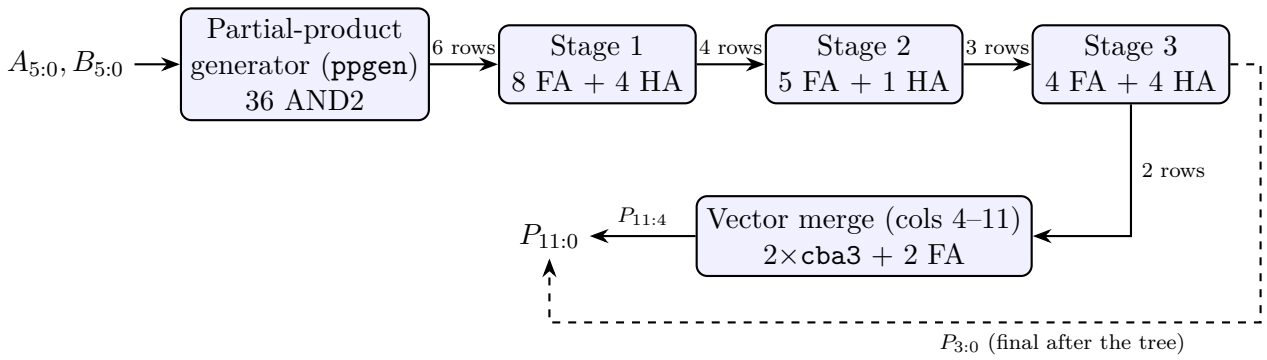


Figure 1: Top-level structure of the multiplier (top cell `wallace6x6`).

Level	Full adders	Half adders	Max. column height (rows)	Bits remaining
Partial products	–	–	6	36
After Stage 1	8	4	4	28
After Stage 2	5	1	3	23
After Stage 3	4	4	2	19
Total	17	9		

Table 2: Row reduction through the Wallace tree. After Stage 3, columns 0–3 hold a single bit each (these are P_0–P_3) and columns 4–11 hold the two rows that the vector merge adds.

3 Basic Building Blocks

3.1 AND2 (`and2`)

Each partial product is an AND function, built as `nand2` followed by `inv`. In layout, well/substrate taps were added inside the cell and the local-interconnect (li1) route was redrawn to meet the latch-up (tap distance) and li1 spacing rules.

3.2 True-output full adder (`fa_true`)

The library full adder is a mirror adder that produces $\overline{S}$ and $\overline{C_{out}}$. Since the Wallace stages and the merge adder need true polarity, `fa_true` is the library `fa` followed by an inverter on each output:

$$S = a \oplus b \oplus c, \qquad C_{out} = ab + bc + ca.$$

3.3 Half adder (`ha`)

As required by the specification, every half adder is a `fa_true` with its third input tied to ground:

$$S = a \oplus b \oplus 0 = a \oplus b, \qquad C_{out} = ab + 0 = ab.$$

Using one cell for both keeps the layout regular and lets all adders share one characterised timing model.

3.4 Naming convention for internal signals

To keep the inter-stage wiring traceable, internal nets follow the pattern

`<type><stage><group>_<column>`

For example, `s1t_3` is a *sum* from *Stage 1*, *top* group, landing in *column 3*, and `c1b_5` is a *carry* from Stage 1, bottom group, landing in column 5.

4 Partial-Product Generator (`ppgen`)

4.1 Function

The first step of any array or tree multiplier is to form every bit-product of the two operands. For two 6-bit unsigned numbers

$$A = \sum_{i=0}^{5} a_i 2^i, \qquad B = \sum_{j=0}^{5} b_j 2^j \quad \Longrightarrow \quad P = A \cdot B = \sum_{j=0}^{5} \sum_{i=0}^{5} (a_i b_j)\, 2^{i+j}.$$

Each product $a_i b_j$ is a single bit (a logical AND) with weight 2^{i+j}. There are $6 \times 6 = 36$ of them. They are arranged in a *dot diagram*: row j is the operand A ANDed with bit b_j, shifted left by j places, so the bit $a_i b_j$ sits in column $k = i + j$. The number of bits in each column is

Column k	0	1	2	3	4	5	6	7	8	9	10	11
Bits	1	2	3	4	5	6	5	4	3	2	1	0

so the tallest column (column 5) holds six bits, and the job of the Wallace tree is to reduce every column to at most two bits.

4.2 Schematic construction

1. An `and2` cell was first built from the library cells as `nand2` → `inv`, with its own symbol (pins `A`, `B`, `Y`, `DVDD`, `DGND`).

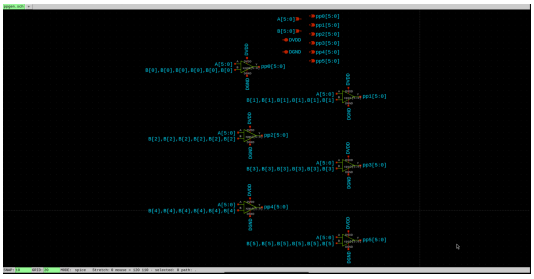
2. In `ppgen.sch` six *instance arrays* of `and2` were placed, `xpp0[5:0]` to `xpp5[5:0]`; one array generates one row.

3. Input `A` of every array is the bus `A[5:0]`. Input `B` of array j is the bit b_j repeated six times (e.g. `B[2],B[2],B[2],B[2],B[2],B[2]`), so the six gates of row j compute $a_5 b_j, \ldots, a_0 b_j$.

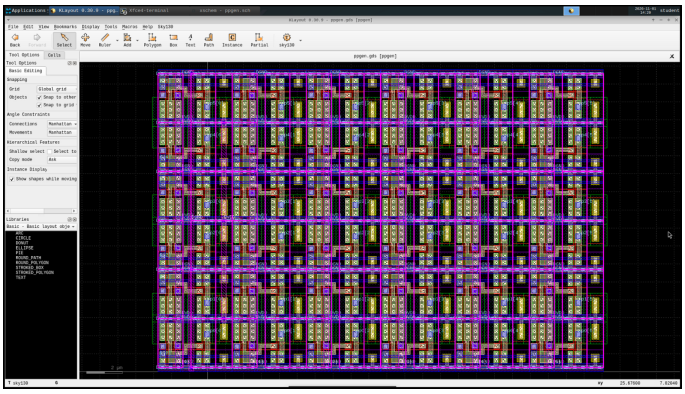
4. The outputs are named `pp`j`[`i`]` $= a_i\, b_j$ and exported as six 6-bit buses `pp0[5:0]` … `pp5[5:0]`. With this naming, `pp`j`[`i`]` belongs to column $i + j$, which makes the wiring of the tree easy to check.

5. The array notation gives exactly 36 AND gates while keeping the schematic readable (Fig. 2).

The bit $a_0 b_0$ (`pp0[0]`) is alone in column 0, so it is the final product bit P_0 and bypasses the whole tree.



(a) Schematic: six `and2` instance arrays



(b) Layout: 6×6 AND array ($25.3 \times 16.8\,\mu$m)

Figure 2: Partial-product generator.

4.3 Layout

The `and2` layout places the `nand2` and `inv` cells side by side on common `met1` power rails. Substrate and n-well taps were added inside the cell, and the `li1` connection between the NAND output and the inverter input was redrawn, because the first version violated the tap-distance (latch-up) and `li1` spacing rules. The 36 gates were then placed as a regular array: six rows of six gates, each row sharing its `DVDD`/`DGND` rails with its neighbour by flipping alternate rows. The 36 outputs are brought to the edge of the array on `met2`/`met3`, where the top-level router picks them up.

5 Wallace Tree Reduction

5.1 Principle

A full adder takes three bits of the same weight and outputs a sum of the same weight and a carry of double weight; it is a 3:2 counter that removes one bit from the tree. A half adder takes two bits and outputs a sum and a carry; it removes no bits but moves one bit to the next column, which is useful for aligning the columns. In a Wallace tree, the rows are taken in groups of three, and every group of three bits in the same column goes into a full adder in parallel. Each stage therefore reduces the number of rows by about 3/2:

$$6 \xrightarrow{\text{Stage 1}} 4 \xrightarrow{\text{Stage 2}} 3 \xrightarrow{\text{Stage 3}} 2.$$

All adders inside one stage work in parallel, so the tree delay is only three adder delays regardless of how many adders are used.

5.2 Common rules followed in every stage

- **Sum stays, carry moves.** An adder placed in column k sends its sum to column k and its carry to column $k+1$ of the next stage.
- **Half adders are full adders.** As required, each HA is the `ha` cell, i.e. a `fa_true` with `Cin` tied to `DGND`.
- **Bits that are not reduced are passed straight through** to the next stage.
- **Net naming.** Outputs are named `<s|c><stage><t|b>_<column>`: e.g. `c1b_7` is the carry of a Stage 1 adder of the *bottom* group that lands in column 7. This makes each net show where it goes, so wiring errors are easy to spot.

- **Each stage is its own subcircuit** (`stage1`, `stage2`, `stage3`) with its own symbol. Inside a stage the adders are connected only by labels, since every adder input comes from the previous stage and every output goes to the next one.

Figure 3 shows the complete reduction, generated directly from the netlists. Each colour is one adder; a bit in the next panel has the colour of the adder that produced it. Full adders have solid outlines and half adders dashed outlines.

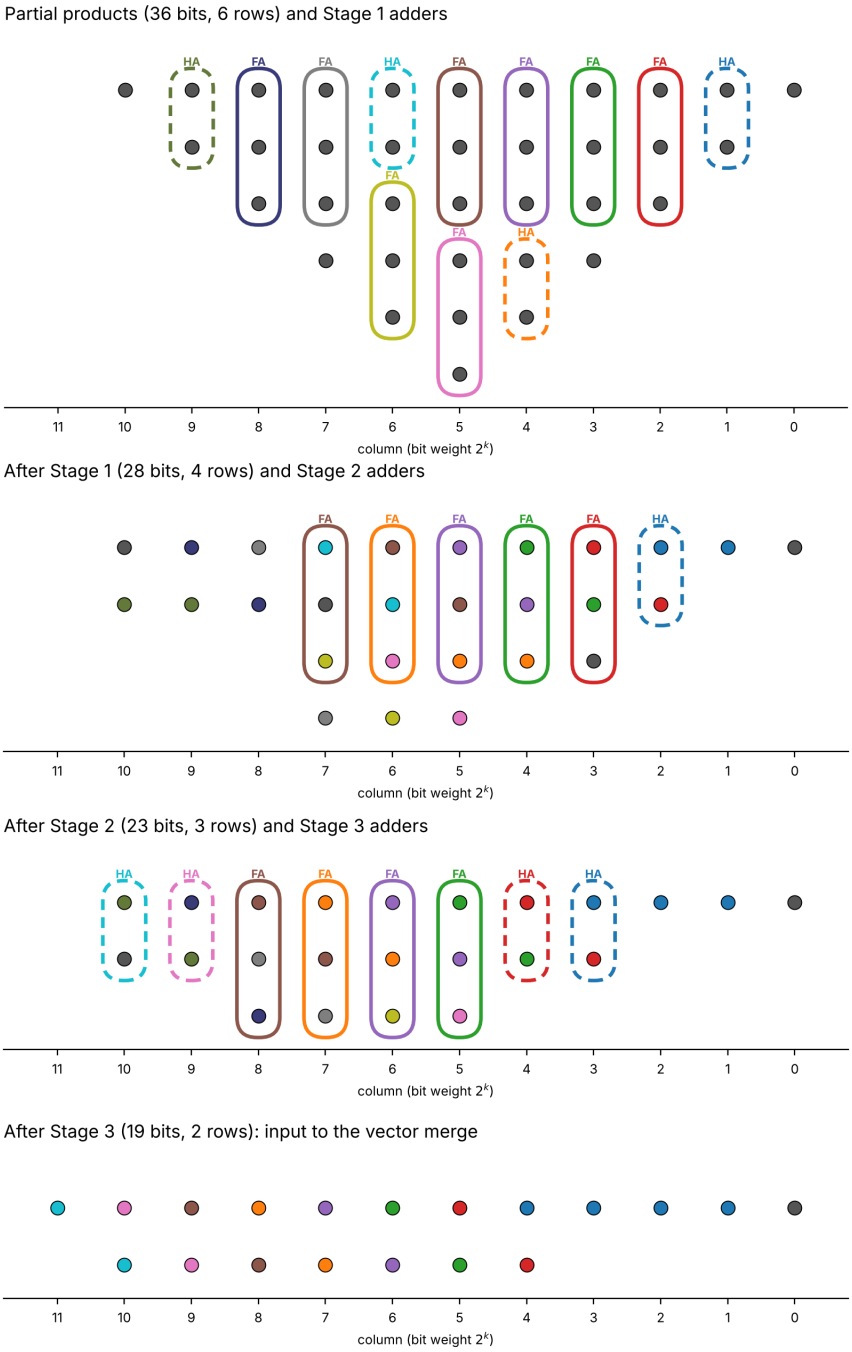


Figure 3: Dot diagram of the full reduction (generated from the stage netlists). Grey dots are partial products; coloured dots are the sum/carry of the adder with the same colour in the previous panel.

5.3 Stage 1: 6 rows → 4 rows

Grouping. The six partial-product rows are split into two groups of three:

- **top group**: rows 0, 1, 2 (b_0, b_1, b_2), columns 0–7;
- **bottom group**: rows 3, 4, 5 (b_3, b_4, b_5), columns 3–10.

Within each group the column heights are $1, 2, 3, 3, 3, 3, 2, 1$. Every column with three bits gets a full adder; a column with two bits gets a half adder.

Procedure.

1. Top group: HA in column 1 (`x1`); FAs in columns 2, 3, 4, 5 (`x2`–`x5`); HA in column 6 (`x6`).
2. Bottom group: HA in column 4 (`x7`); FAs in columns 5, 6, 7, 8 (`x8`–`x11`); HA in column 9 (`x12`).
3. The single bits at the ends of each group are passed through: a_0b_0 (column 0), a_0b_3 (column 3), a_5b_2 (column 7) and a_5b_5 (column 10).

Stage 1 therefore uses **8 FAs and 4 HAs** and leaves 28 bits in at most 4 rows. Column 1 now holds a single bit, `s1t_1`, which is the final product bit P_1. Table 3 lists every adder, taken from `stage1.spice`.

Inst.	Type	Col.	Inputs	Sum (col k)	Carry (col k+1)
`x1`	HA	1	a_1b_0, a_0b_1	`s1t_1`	`c1t_2`
`x2`	FA	2	a_2b_0, a_1b_1, a_0b_2	`s1t_2`	`c1t_3`
`x3`	FA	3	a_3b_0, a_2b_1, a_1b_2	`s1t_3`	`c1t_4`
`x4`	FA	4	a_4b_0, a_3b_1, a_2b_2	`s1t_4`	`c1t_5`
`x7`	HA	4	a_1b_3, a_0b_4	`s1b_4`	`c1b_5`
`x5`	FA	5	a_5b_0, a_4b_1, a_3b_2	`s1t_5`	`c1t_6`
`x8`	FA	5	a_2b_3, a_1b_4, a_0b_5	`s1b_5`	`c1b_6`
`x6`	HA	6	a_5b_1, a_4b_2	`s1t_6`	`c1t_7`
`x9`	FA	6	a_3b_3, a_2b_4, a_1b_5	`s1b_6`	`c1b_7`
`x10`	FA	7	a_4b_3, a_3b_4, a_2b_5	`s1b_7`	`c1b_8`
`x11`	FA	8	a_5b_3, a_4b_4, a_3b_5	`s1b_8`	`c1b_9`
`x12`	HA	9	a_5b_4, a_4b_5	`s1b_9`	`c1b_10`

Table 3: Stage 1 adders (from the netlist). a_ib_j is `pp`j`[`i`]`.

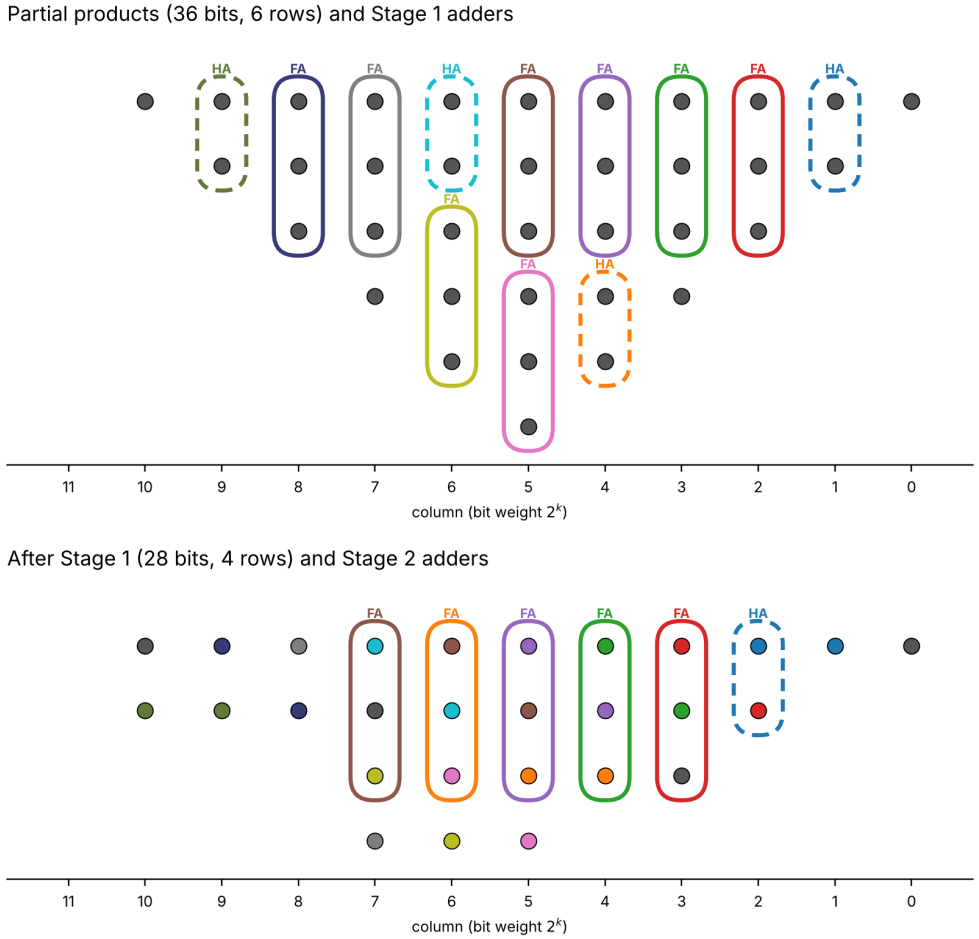


Figure 4: Stage 1: the 12 adders acting on the partial products (top) and the 4-row result (bottom).

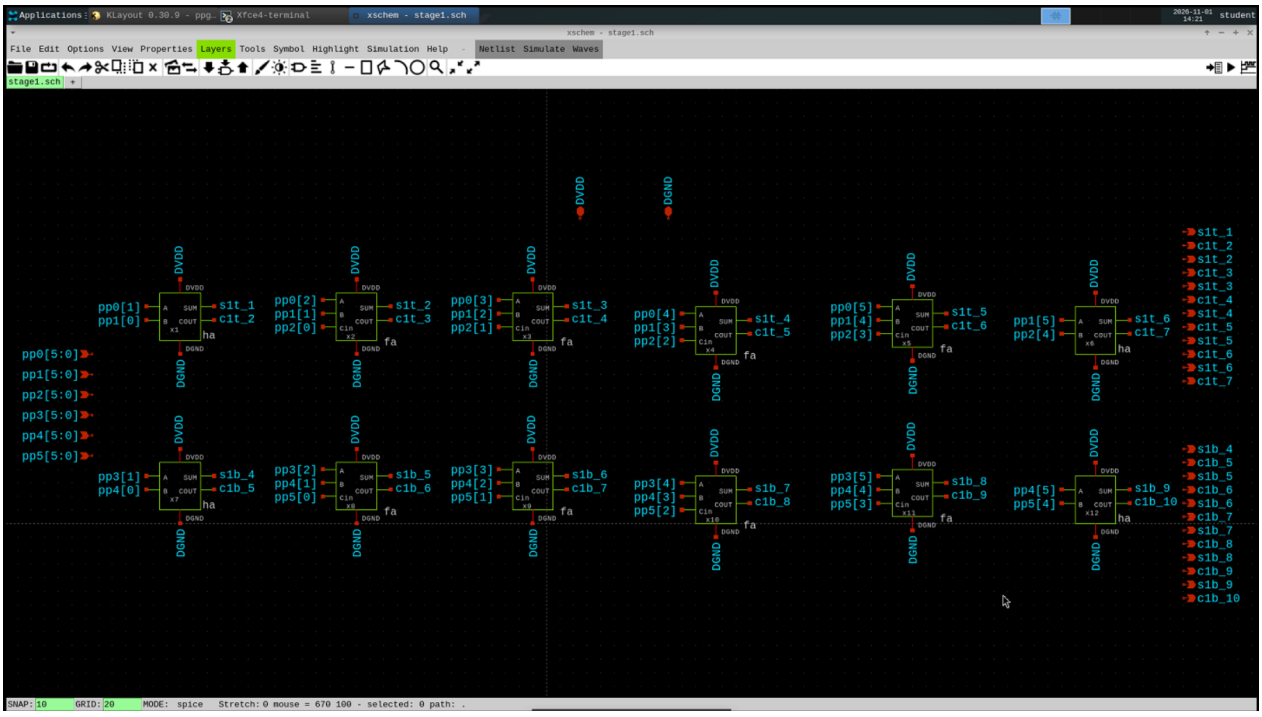


Figure 5: Stage 1 schematic (`stage1.sch`): top-group adders `x1`–`x6` in the upper row, bottom-group adders `x7`–`x12` in the lower row.

5.4 Stage 2: 4 rows → 3 rows

Grouping. After Stage 1 the column heights are $1, 1, 2, 3, 3, 4, 4, 4, 2, 2, 2$ (columns 0–10). The outputs of the top group, together with the two pass-through bits a_0b_3 and a_5b_2 and the low-order outputs of the bottom group, form the three rows reduced in this stage; the remaining bottom-group outputs (the fourth row) wait for Stage 3.

Procedure.

1. Column 2 has exactly two bits (`c1t_2`, `s1t_2`): one HA (`x1`). Its sum `s2_2` is the final product bit P_2.
2. Columns 3–7 each get one FA (`x2`–`x6`). The FA in column 4, for example, adds `c1t_4`, `s1t_4` and `s1b_4`; the FA in column 7 adds `c1t_7`, a_5b_2 and `c1b_7`.
3. Passed through to Stage 3: `s1b_5`, `s1b_6`, `s1b_7`, `c1b_8`, `s1b_8`, `c1b_9`, `s1b_9`, `c1b_10` and a_5b_5 (plus the final bits P_0 and P_1).

Stage 2 uses **5 FAs and 1 HA** and leaves 23 bits in at most 3 rows.

Inst.	Type	Col.	Inputs	Sum (col k)	Carry (col k+1)
`x1`	HA	2	`c1t_2`, `s1t_2`	`s2_2`	`c2_3`
`x2`	FA	3	`c1t_3`, `s1t_3`, a_0b_3	`s2_3`	`c2_4`
`x3`	FA	4	`c1t_4`, `s1t_4`, `s1b_4`	`s2_4`	`c2_5`
`x4`	FA	5	`c1t_5`, `s1t_5`, `c1b_5`	`s2_5`	`c2_6`
`x5`	FA	6	`c1t_6`, `s1t_6`, `c1b_6`	`s2_6`	`c2_7`
`x6`	FA	7	`c1t_7`, a_5b_2, `c1b_7`	`s2_7`	`c2_8`

Table 4: Stage 2 adders (from the netlist).

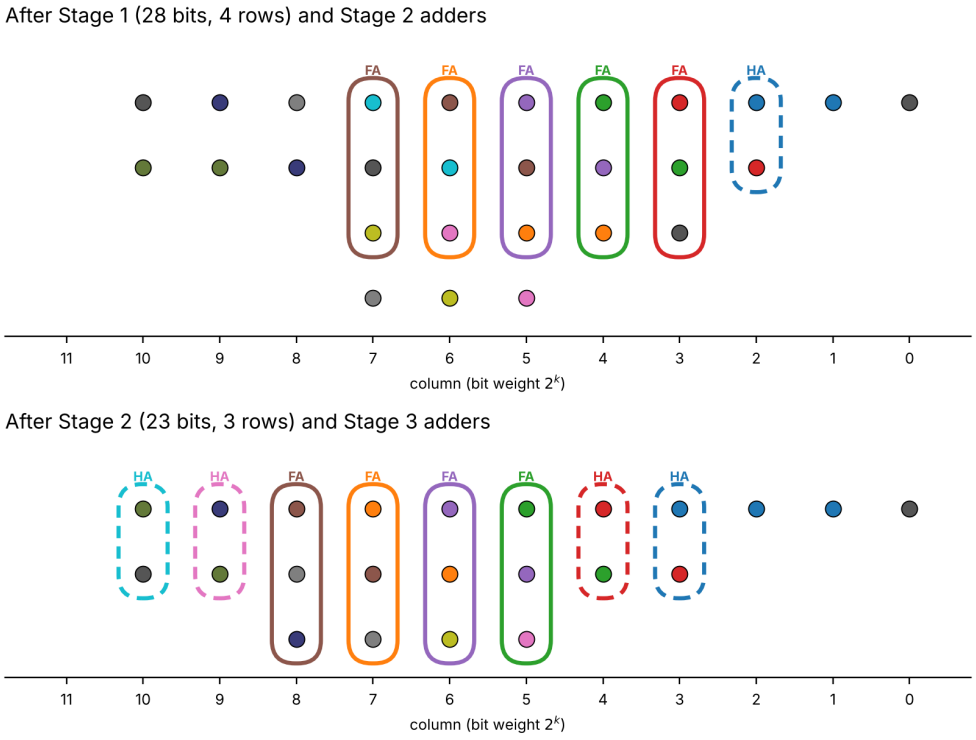


Figure 6: Stage 2: one HA and five FAs reduce the 4-row array to 3 rows.

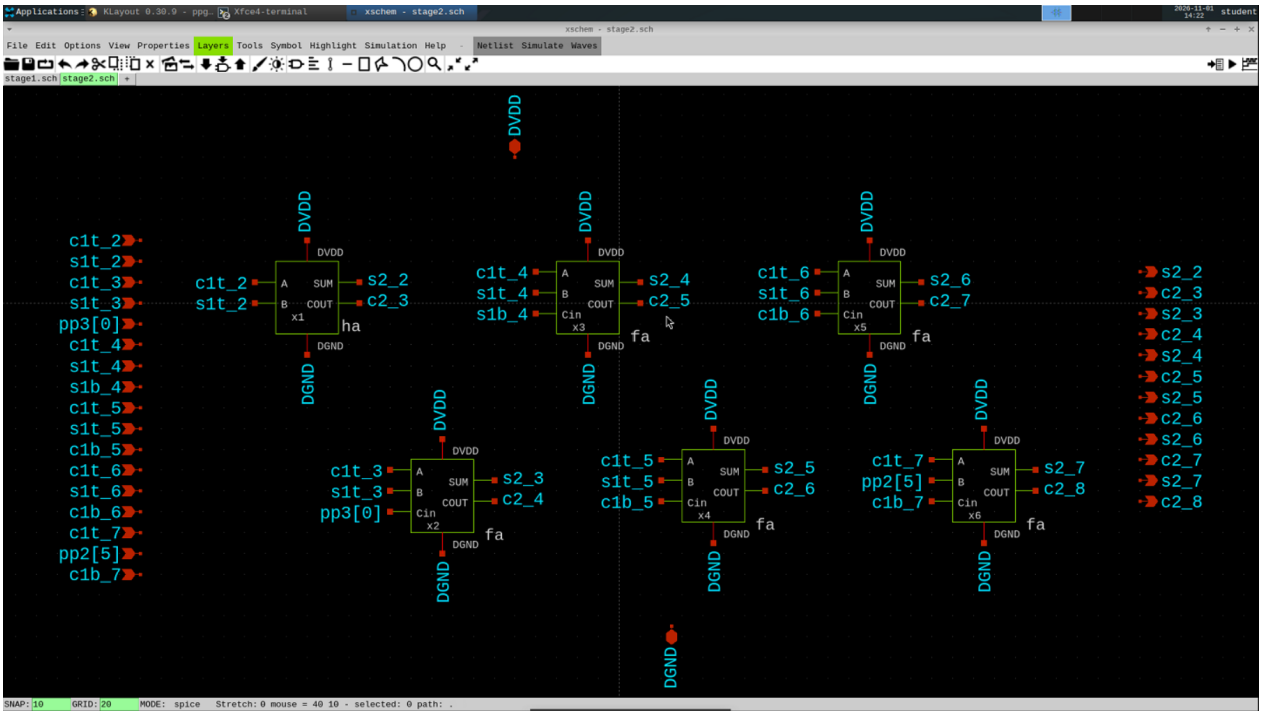


Figure 7: Stage 2 schematic (`stage2.sch`).

5.5 Stage 3: 3 rows → 2 rows

Procedure. After Stage 2 the column heights are $1, 1, 1, 2, 2, 3, 3, 3, 3, 2, 2$.

1. Columns with three bits (5, 6, 7, 8) get a FA each (`x2`, `x3`, `x5`, `x6`). Here the Stage 2 outputs meet the bottom-group bits that were held back, e.g. column 5 adds `c2_5`, `s2_5` and `s1b_5`.

2. Columns 3, 4, 9 and 10 have two bits each and get a HA (`x1`, `x4`, `x7`, `x8`). The HA in column 3 produces `s3_3`, the final product bit P_3. The HAs in columns 4, 9 and 10 are not strictly needed to reach two rows, but they shift a bit into the next column so that the two final rows are aligned (column 11 receives the carry `c3_11`).

Stage 3 uses **4 FAs and 4 HAs**. Its outputs form the two operand rows of the vector merge:

$$X_k = \texttt{s3_}k \ (k = 4 \ldots 10), \quad X_{11} = \texttt{c3_11}, \qquad Y_k = \texttt{c3_}k \ (k = 4 \ldots 10).$$

Columns 0–3 are already final ($P_0 = a_0 b_0$, $P_1 = \texttt{s1t_1}$, $P_2 = \texttt{s2_2}$, $P_3 = \texttt{s3_3}$).

Inst.	Type	Col.	Inputs	Sum (col k)	Carry (col k+1)
x1	HA	3	c2_3, s2_3	s3_3	c3_4
x4	HA	4	c2_4, s2_4	s3_4	c3_5
x2	FA	5	c2_5, s2_5, s1b_5	s3_5	c3_6
x3	FA	6	c2_6, s2_6, s1b_6	s3_6	c3_7
x5	FA	7	c2_7, s2_7, s1b_7	s3_7	c3_8
x6	FA	8	c2_8, c1b_8, s1b_8	s3_8	c3_9
x7	HA	9	c1b_9, s1b_9	s3_9	c3_10
x8	HA	10	c1b_10, $a_5 b_5$	s3_10	c3_11

Table 5: Stage 3 adders (from the netlist).

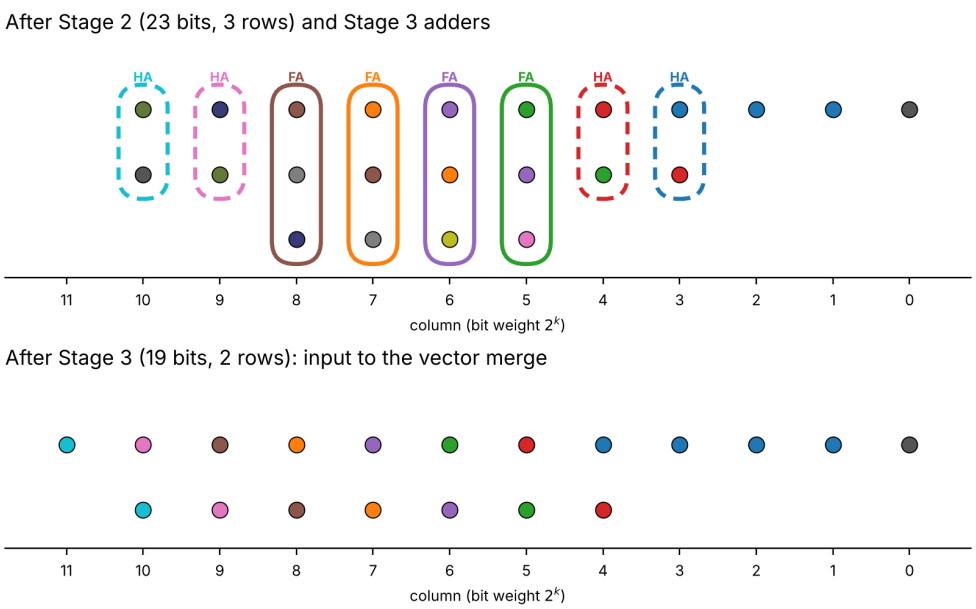


Figure 8: Stage 3: four FAs and four HAs leave exactly two rows (columns 4–11).

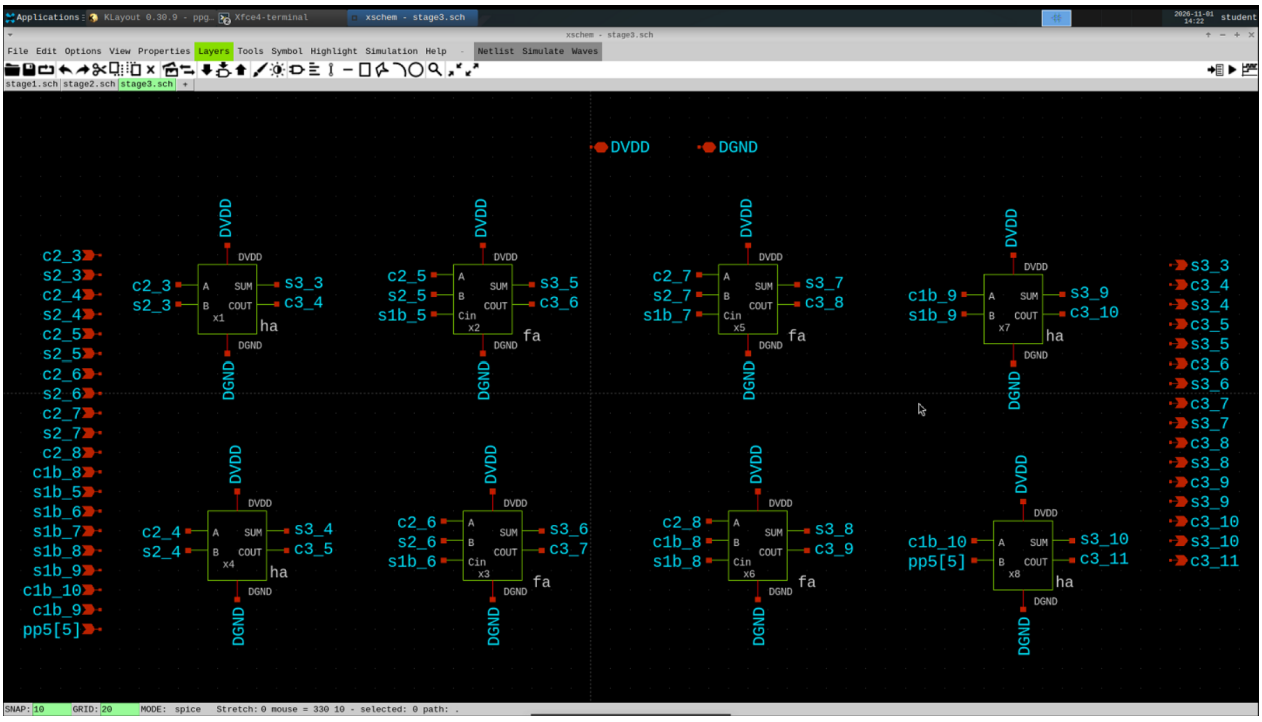


Figure 9: Stage 3 schematic (`stage3.sch`).

5.6 Summary of the tree

	FA	HA	Rows after	Bits after	Final bits produced
Stage 1	8	4	4	28	P_0 (pass), P_1
Stage 2	5	1	3	23	P_2
Stage 3	4	4	2	19	P_3
Total	17	9			

Table 6: Adders used in the tree. In total 26 `fa_true` cells (9 of them used as HAs).

6 Vector Merge: 3-bit Carry-Bypass Adder

6.1 Why a carry-bypass adder

The last two rows, $X_{11:4}$ and $Y_{10:4}$, must be added by a two-operand adder. The specification asks for a ripple-carry adder and gives a bonus for a faster one. In a ripple adder the carry passes through every bit, so its delay is

$$t_{RCA} \approx (n-1)\, t_{carry} + t_{sum}.$$

A carry-bypass (carry-skip) adder divides the bits into blocks. For each block it computes the propagate signals $p_i = x_i \oplus y_i$. If all of them are 1, the block's carry-out equals its carry-in, so the carry-in can skip the block through a multiplexer instead of rippling through it. For blocks of M bits:

$$t_{CBA} \approx t_{setup} + M\, t_{carry} + \left(\tfrac{n}{M} - 1\right) t_{mux} + (M-1)\, t_{carry} + t_{sum}.$$

6.2 Choice of 3-bit blocks

The merge spans 8 columns (4–11), split as $3 + 3 + 2$:

- **Block 1** (`cba3`, instance `x1`): columns 4, 5, 6, with $C_{in} = 0$;
- **Block 2** (`cba3`, instance `x2`): columns 7, 8, 9;
- **Column 10**: `fa_true` (`x3`) adding X_{10}, Y_{10} and block 2's carry-out;
- **Column 11**: `fa_true` (`x4`) with one input tied to 0 (only X_{11} and the column-10 carry); its sum is P_{11}. Its carry-out is always 0, because $63 \times 63 = 3969 < 2^{12}$, and is left unconnected.

The carry into block 1 is constant (0), so block 1's bypass multiplexer can never speed up a carry *entering* the block. Its value is in the multiplexer of block 2: a carry generated in block 1 can skip the whole of block 2. With 4-bit blocks there would be only one useful bypass, so 3-bit blocks, as suggested in the problem statement, were used.

6.3 The cba3 block

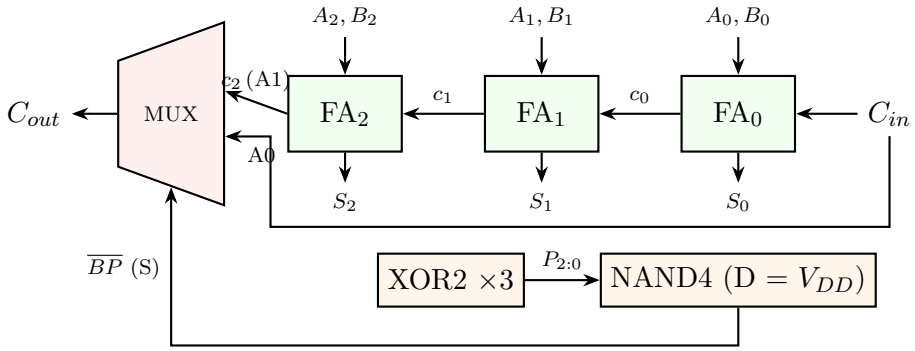


Figure 10: Structure of the 3-bit carry-bypass block `cba3`, as in `cba3.sch`.

Construction (Fig. 10, 11):

1. **Ripple chain.** An instance array `x[2:0]` of `fa_true` with inputs `A[2:0]`, `B[2:0]` and carry-in bus `c[1:0],Cin`, and outputs `S[2:0]` and carries `c[2:0]`. The bus notation chains the carries: FA$_0$ takes `Cin`, FA$_1$ takes `c[0]`, FA$_2$ takes `c[1]`, and `c[2]` is the rippled block carry.

2. **Propagate signals.** An instance array `x1[2:0]` of `xor2` gives $P_i = A_i \oplus B_i$.

3. **Block propagate.** The library `nand4` combines P_0, P_1, P_2, with its fourth input tied to `DVDD`:

$$\texttt{BPn} = \overline{P_0\, P_1\, P_2}.$$

   Using the active-low signal directly as the multiplexer select avoids an extra inverter.

4. **Bypass multiplexer.** The library `mux2` has `S`=`BPn`, `A0`=`Cin`, `A1`=`c[2]`:

$$C_{out} = \begin{cases} C_{in}, & \texttt{BPn} = 0 \text{ (all three bits propagate: bypass)} \\ c_2, & \texttt{BPn} = 1 \text{ (the carry is generated or killed inside the block).} \end{cases}$$

   When the block propagates, $c_2 = C_{in}$ anyway, so the multiplexer never changes the result; it only gives a faster path.

Design pitfall that was fixed. The first version used the library `fa` cell directly. Because `fa` outputs $\overline{S}$ and $\overline{C_{out}}$, chaining it like a normal adder gave wrong results. Exhaustive simulation exposed this, and the block was rebuilt with `fa_true`.

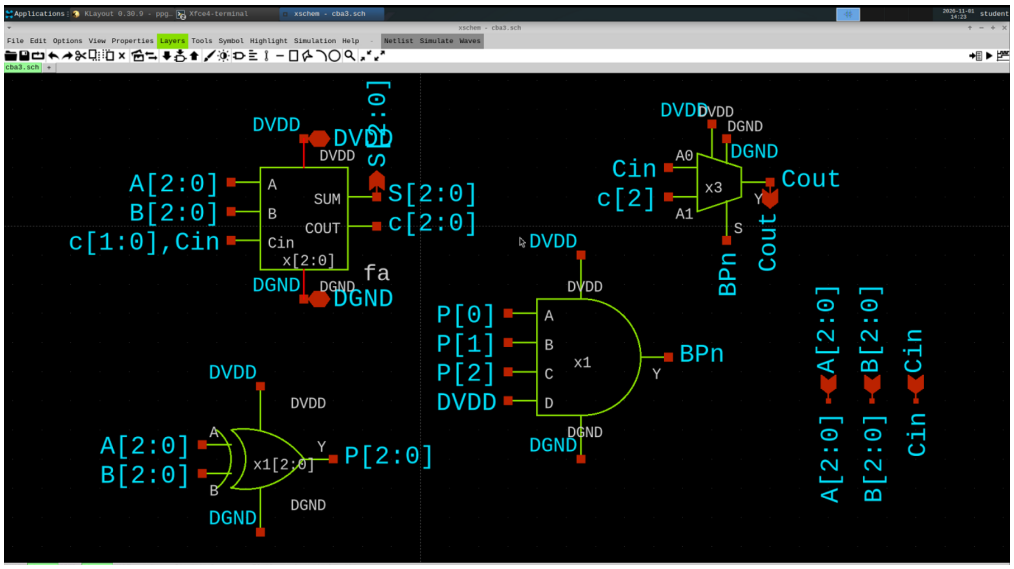


Figure 11: `cba3` schematic: FA array (top left), XOR array (bottom left), `nand4` (centre) and `mux2` (top right).

6.4 Complete vector merge (`vectormerge`)

1. Two `cba3` instances: `x1` on `X[6:4]`, `Y[6:4]` with `Cin`=`DGND`; `x2` on `X[9:7]`, `Y[9:7]` with `Cin` = `x1.Cout`.
2. `x3` (`fa_true`) adds `X[10]`, `Y[10]` and `x2.Cout`.
3. `x4` (`fa_true`) adds `X[11]`, 0 and the column-10 carry.
4. The sum bus `S[11:4]` is the product $P_{11:4}$.

The merge was verified on its own over all input combinations before it was connected to the tree.

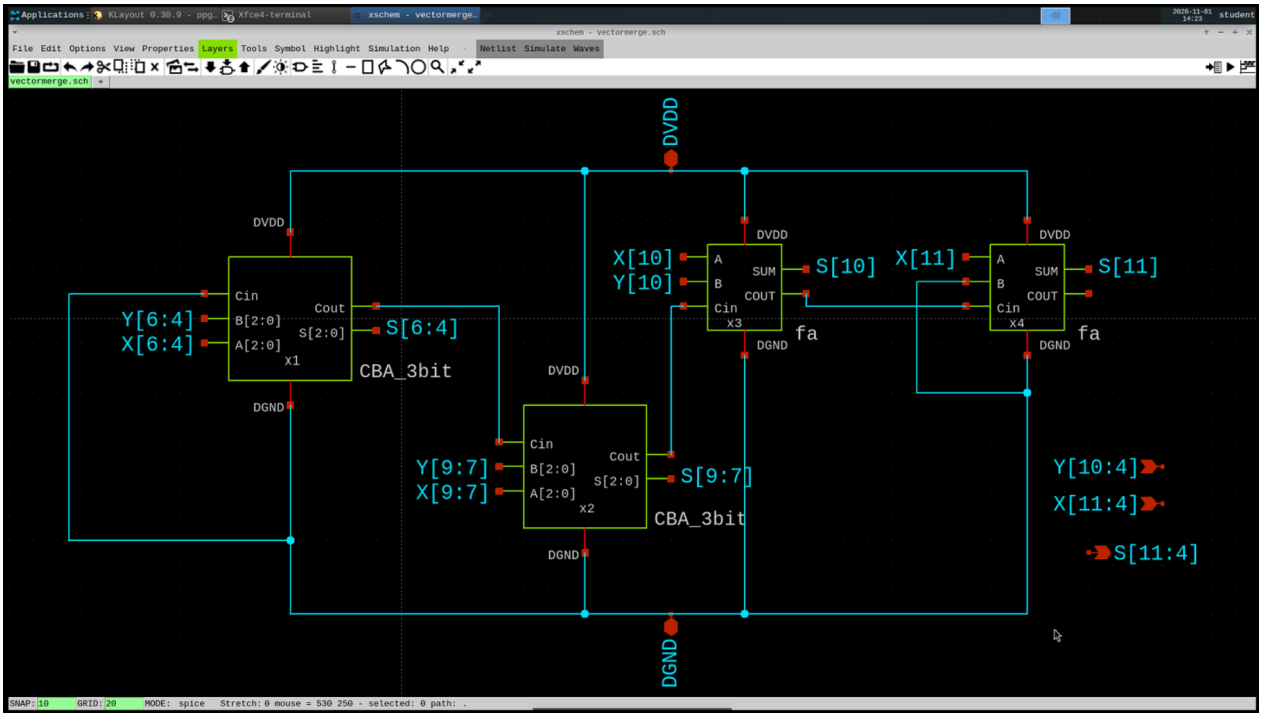


Figure 12: `vectormerge` schematic: two `cba3` blocks and two full adders.

7 Top-Level Schematic and Functional Verification

7.1 Top-level schematic

The top-level schematic connects `ppgen`, `stage1`, `stage2`, `stage3` and `vectormerge` through their symbols. Its inputs are `A[5:0]`, `B[5:0]`, `DVDD` and `DGND`, and its output is `P[11:0]`. The connections are:

- `ppgen` → all three stages (each stage takes the partial products it needs, by name) and P_0;
- `stage1` → `stage2` (top-group outputs and `s1b_4`, `c1b_5..7`) and → `stage3` (the held-back bottom-group outputs); `s1t_1` → P_1;
- `stage2` → `stage3`; `s2_2` → P_2;
- `stage3` → `vectormerge` as $X_{11:4}$, $Y_{10:4}$; `s3_3` → P_3;
- `vectormerge` → $P_{11:4}$.

7.2 Verification procedure

1. **Block-level exhaustive checks.** After each block was drawn, its xschem netlist was converted into a gate-level logic model and evaluated for every input combination: 2^7 for `cba3`, all combinations of the merge inputs, and all $2^{12} = 4096$ combinations of (A, B) for each stage added on top of the previous ones. The output was compared with the arithmetic result ($A \times B$, or the expected column sums).

2. **Errors found and fixed this way:**
   - `cba3`: the inverted-output `fa` cell had been used; replaced by `fa_true`;
   - Stage 2: one adder had two outputs shorted together;
   - Stage 3: one adder input referenced the wrong signal;
   - top level: the connection `s1b_4` from Stage 1 to Stage 2 was missing.

3. **Full multiplier.** After the fixes, the complete netlist gave the correct product for all 4096 input pairs.

4. **Transistor-level check.** ngspice transient simulations of the schematic (SKY130 `tt`, 1.8 V) were run with input sequences such as 81 → 513 → 81 and checked for correct, full-swing outputs.

8 Layout

8.1 Design flow

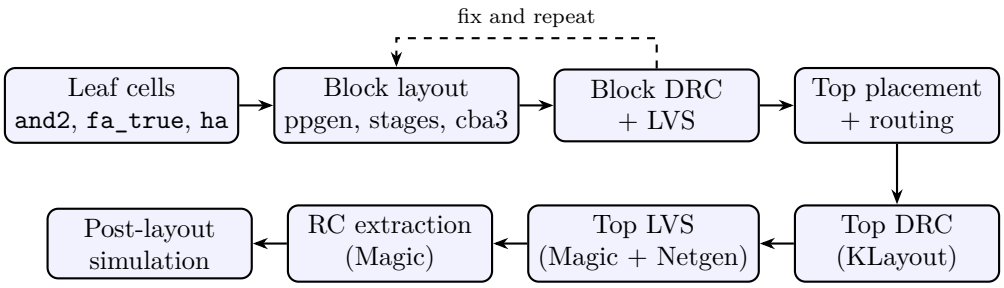


Figure 14: Layout and verification flow used for every block and for the top level.

The layout was built bottom-up, following the schematic hierarchy exactly, so that each layout cell has a schematic cell with the same name and pin list for LVS:

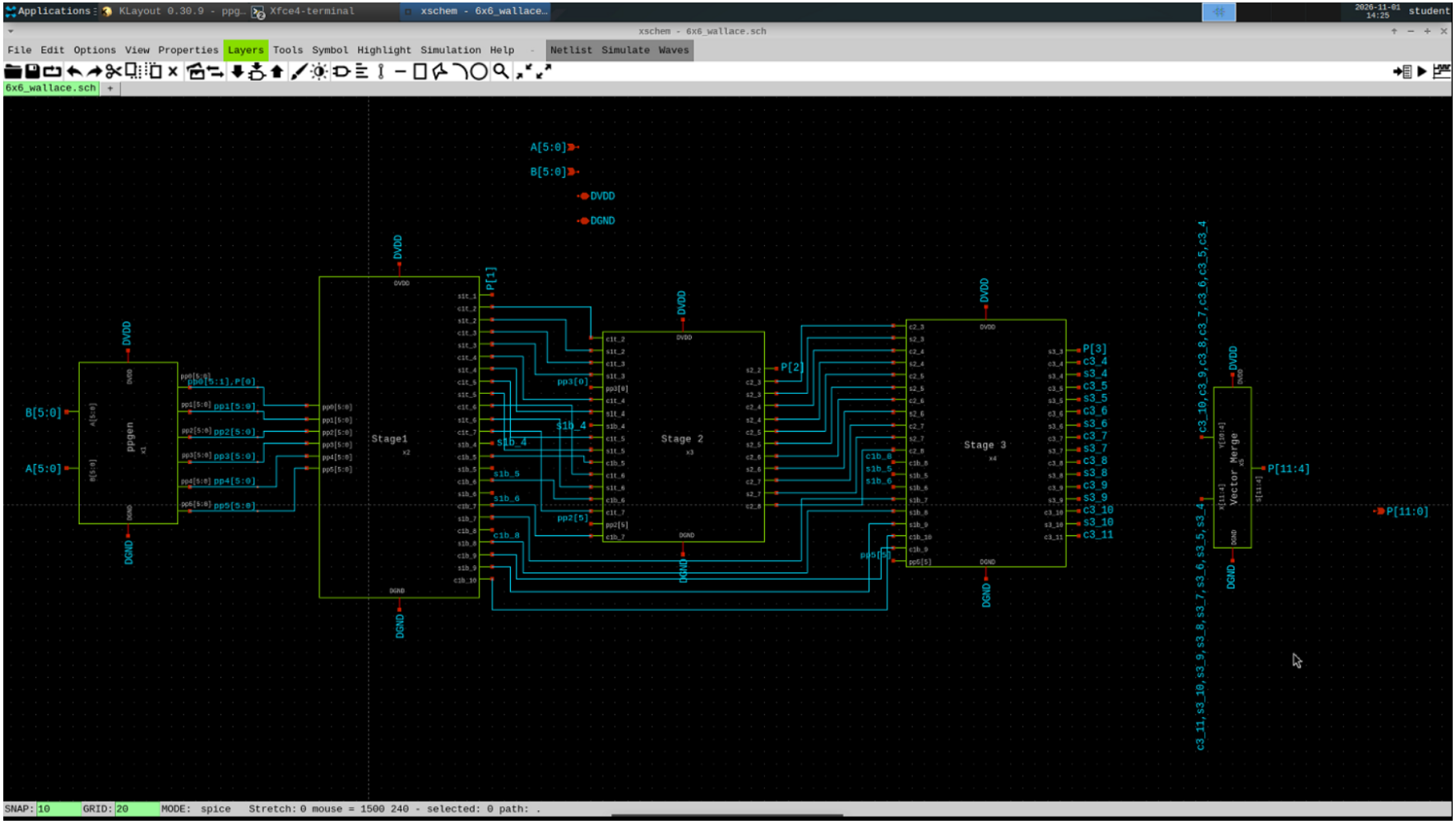


Figure 13: Top-level schematic `6x6_wallace.sch` (cell renamed `wallace6x6` for the layout flow). Left to right: `ppgen`, `stage1`, `stage2`, `stage3`, `vectormerge`.

1. **Leaf cells.** `fa_true` is the library `fa` with two `inv` cells abutted on its `SUMB`/`COUTB` outputs. `ha` is `fa_true` with `Cin` strapped to the ground rail. A small `tap` cell (substrate and n-well contacts) is placed between adders so that no transistor is farther from a tap than the latch-up rule allows. Wrapper cells `fa_w`/`ha_w` contain the adder with its taps and pin access.

2. **Row structure.** All cells share the same height and the same `met1` `DVDD` (top) and `DGND` (bottom) rails, so cells are abutted in rows like standard cells and the rails connect automatically.

3. **Blocks.**

   - `ppgen`: a 6×6 array of `and2` ($25.3 \times 16.8\,\mu$m);
   - `stage1`: two rows of six adders, top and bottom group ($64.0 \times 10.4\,\mu$m);
   - `stage2`: one row of six adders ($64.0 \times 3.2\,\mu$m);
   - `stage3`: two rows of four adders ($42.8 \times 10.4\,\mu$m);
   - `cba3`: 3 FAs, 3 XORs, NAND4 and MUX in one row ($49.2 \times 9.3\,\mu$m);
   - `vectormerge`: the two `cba3` blocks in two rows, plus the column-10/11 adders, with internal carries on `met2`/`met3` ($70.4 \times 10.4\,\mu$m).

   Adders are placed in column order (low column at the left), so a sum goes to the adder directly above and a carry to its left neighbour, keeping wires short.

4. **Top level.** The blocks are stacked bottom to top in signal-flow order: `ppgen`, Stage 1, Stage 2, Stage 3, vector merge (Fig. 15). About 84 inter-block nets were routed on the upper metals: `met3` for horizontal tracks and `met4` for vertical tracks, with stacked vias down to the cell pins. Keeping `li1`–`met2` for wiring inside cells and blocks means the top-level routes never interfere with the cells underneath. The final layout measures $71.2 \times 75.4\,\mu$m.

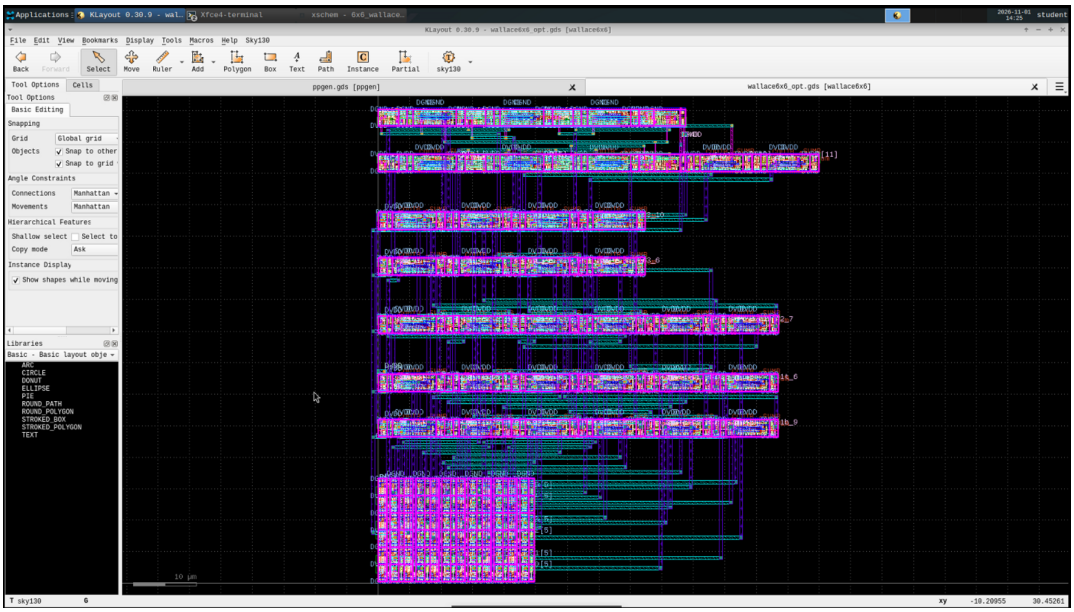
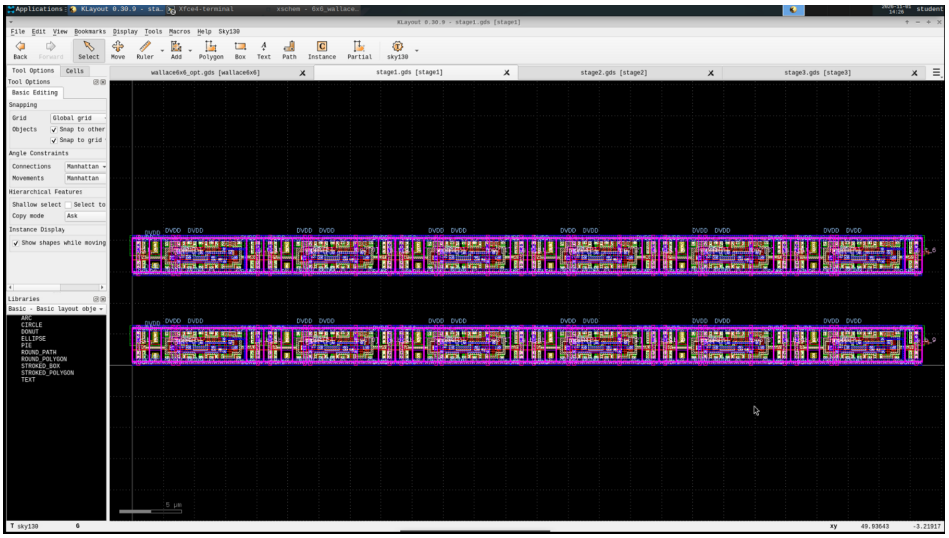
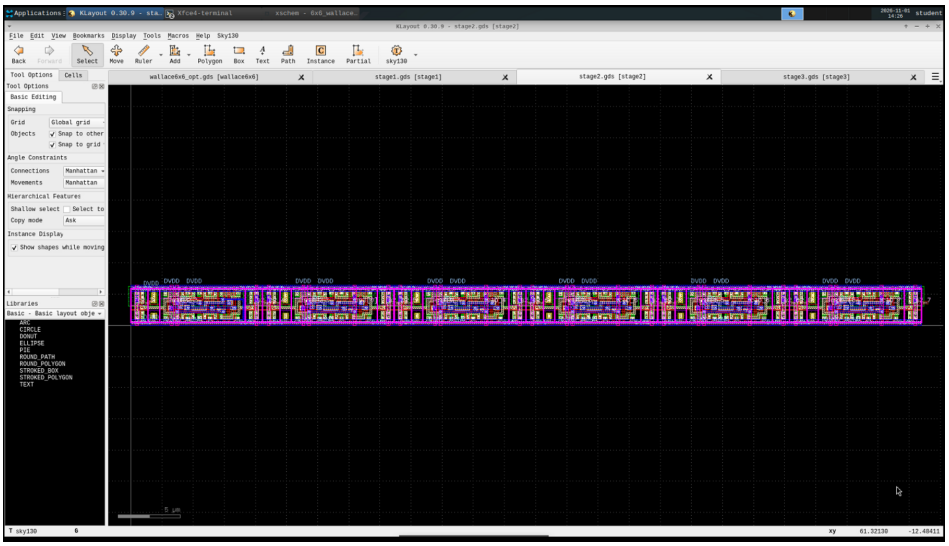


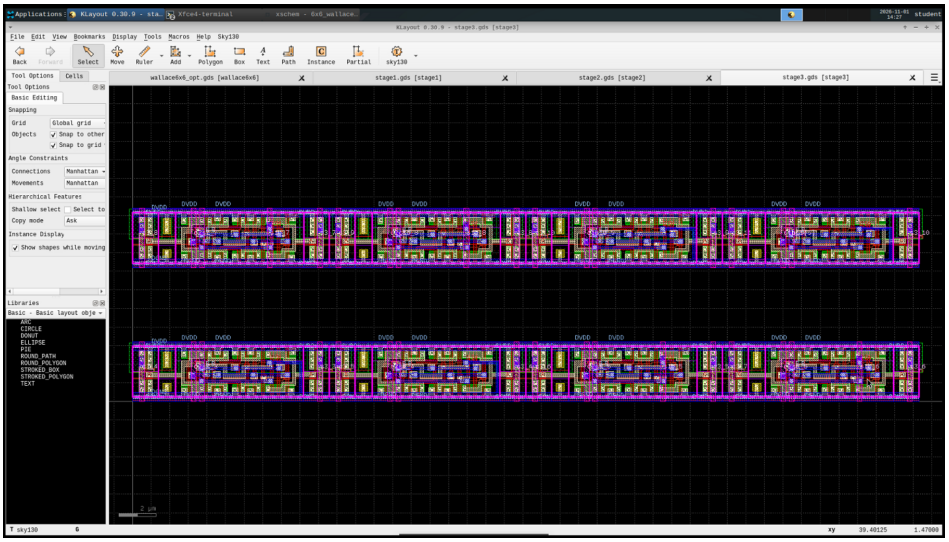
Figure 15: Final top-level layout `wallace6x6` ($71.2 \times 75.4\,\mu$m) in KLayout. From bottom to top: `ppgen`, Stage 1 (two rows), Stage 2, Stage 3 (two rows) and the vector merge (two rows); inter-block nets run on `met3` (horizontal) and `met4` (vertical).



(a) Stage 1: top group (upper row), bottom group (lower row)

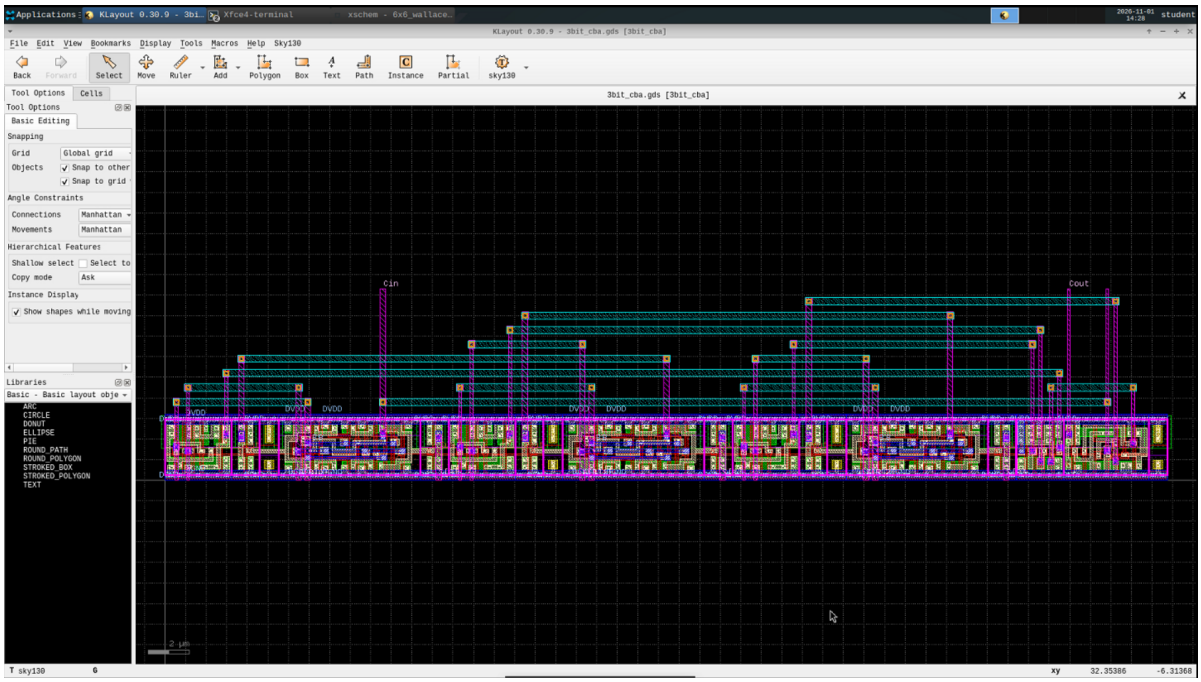


(b) Stage 2: one row of six adders

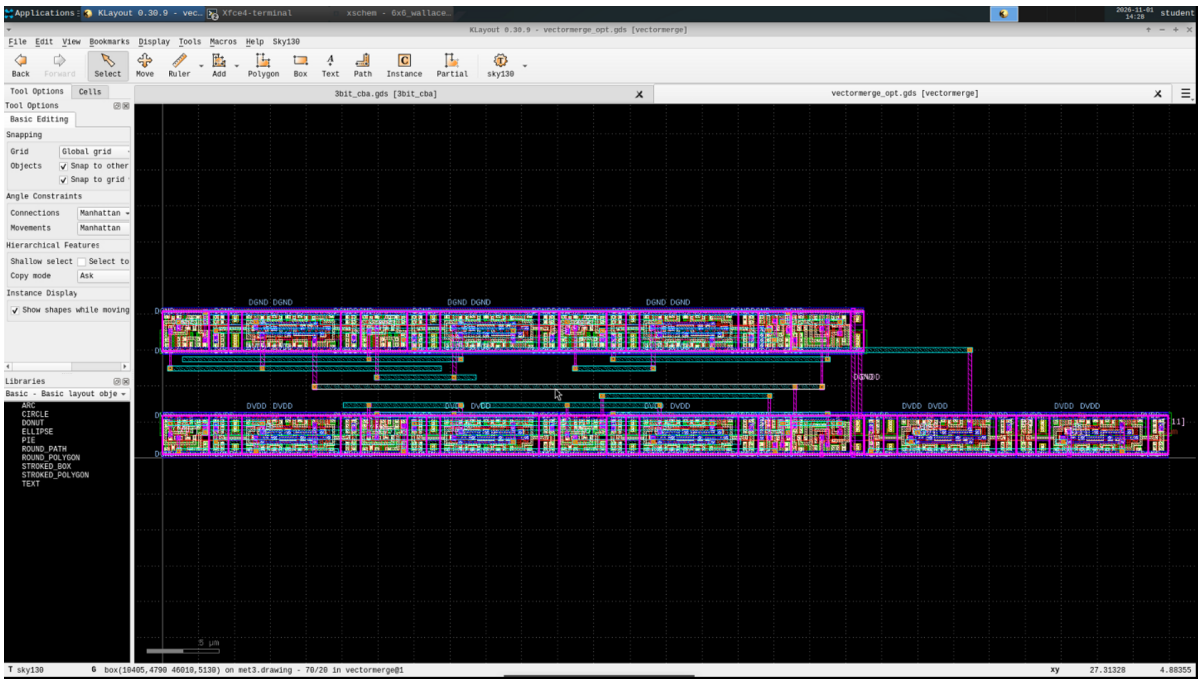


(c) Stage 3: two rows of four adders

Figure 16: Layouts of the three Wallace-tree stages.



(a) `cba3` block



(b) Complete vector merge

Figure 17: Vector-merge layouts.

8.2 Issues found and fixed during layout

1. **Latch-up and `li1` spacing in `and2`.** The first `and2` had no taps and an `li1` route too close to a neighbouring shape. Taps were added and the route redrawn; the cell was made DRC-clean before being arrayed.

2. **Cell names starting with a digit.** `3bit_cba` and `6x6_wallace` broke Magic/Netgen, which treat such names as numbers. They were renamed `cba3` and `wallace6x6` in both schematic and layout.

3. **Comma-separated pin lists.** xschem wrote some bus pins as comma lists in the `.subckt` line,

which the LVS netlist reader could not parse. They were expanded into individual pins.

4. **`mux2` schematic vs. layout mismatch.** Transistors M5–M8, M11 and M12 of the supplied `mux2` schematic did not match the supplied layout. They were set to $W = 0.42\,\mu$m so that LVS matched device sizes.

5. **Extraction script.** Magic's `readspice` (used to set the port order) leaves the last subcircuit of the file loaded, which was `mux2`. The script was changed to reload the real top cell before flattening and extracting.

9 Physical Verification: DRC and LVS

9.1 Design rule check

Procedure.

1. DRC was run in KLayout with the full SKY130 rule deck (macro `run_drc_full.lydrc`), first on every leaf cell and block, then on the top cell `wallace6x6` of `wallace6x6_opt.gds`.
2. Violations were inspected in the marker browser, fixed in the layout, and DRC was run again until no markers remained.

Result. The report `sky130_drc.txt` checks **269 rule categories** (well, diffusion, poly, `licon`, `li1`, `mcon`, `met1`–`met4`, vias, latch-up, density and others) and contains **0 violations**.

Listing 1: Header of the KLayout DRC report (`sky130_drc.txt`).

```
<description>SKY130 DRC runset</description>
<original-file>.../wallace6x6_opt.gds</original-file>
<generator>drc: script='.../run_drc_full.lydrc'</generator>
<top-cell>wallace6x6</top-cell>
...  269 rule categories, 0 items (no violations)
```

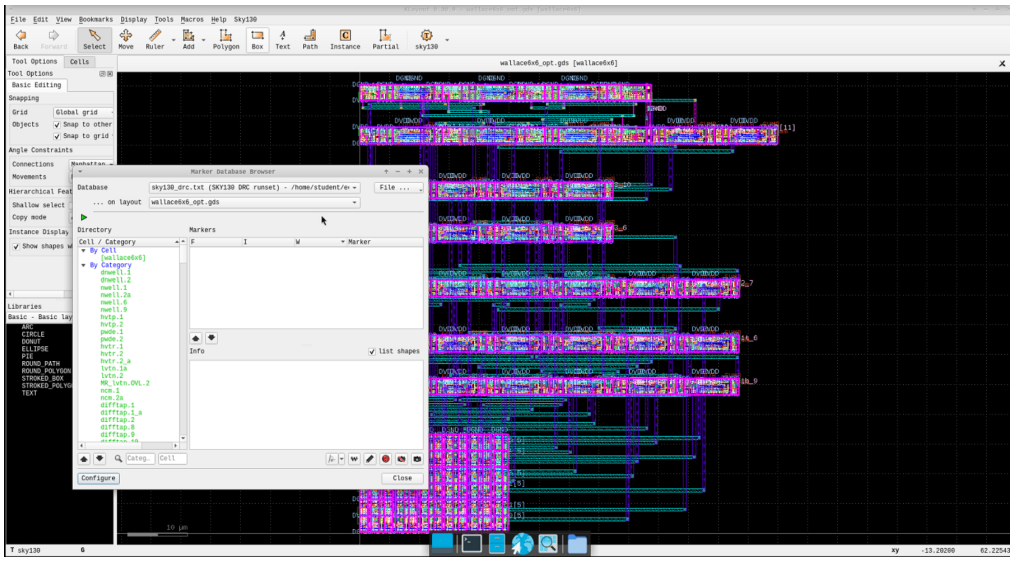


Figure 18: KLayout DRC result for the top-level layout: no violations.

9.2 Layout versus schematic

Procedure.

1. Each block was checked by LVS on its own (`and2`, `ha`, `fa_true`, `ppgen`, `stage1`–`3`, `cba3`, `vectormerge`) before being used in the next level.

2. For the top level, Magic read the GDS, extracted the devices and connectivity, and wrote `wallace6x6_lvs.spice`.
3. Netgen compared this netlist with the xschem netlist of `wallace6x6`, using the SKY130 setup file so that `nfet_01v8`/`pfet_01v8` devices are compared with their W and L.

Result (`comp.out`): Netgen first confirms that the `nfet_01v8` and `pfet_01v8` device classes are equivalent, flattens the sub-cells (`ppgen`, the 36 `and2`, the stages, `cba3`, `xor2`, `nand4`, `mux2`, ...) into `wallace6x6`, and then finds both circuits identical in size and connectivity:

Listing 2: Summary of the Netgen LVS report (`comp.out`).

```
Circuit 1: wallace6x6                    |Circuit 2: wallace6x6
Number of devices: 1268                  |Number of devices: 1268
Number of nets: 650                      |Number of nets: 650
Netlists match uniquely.
Final result: Circuits match uniquely.
```

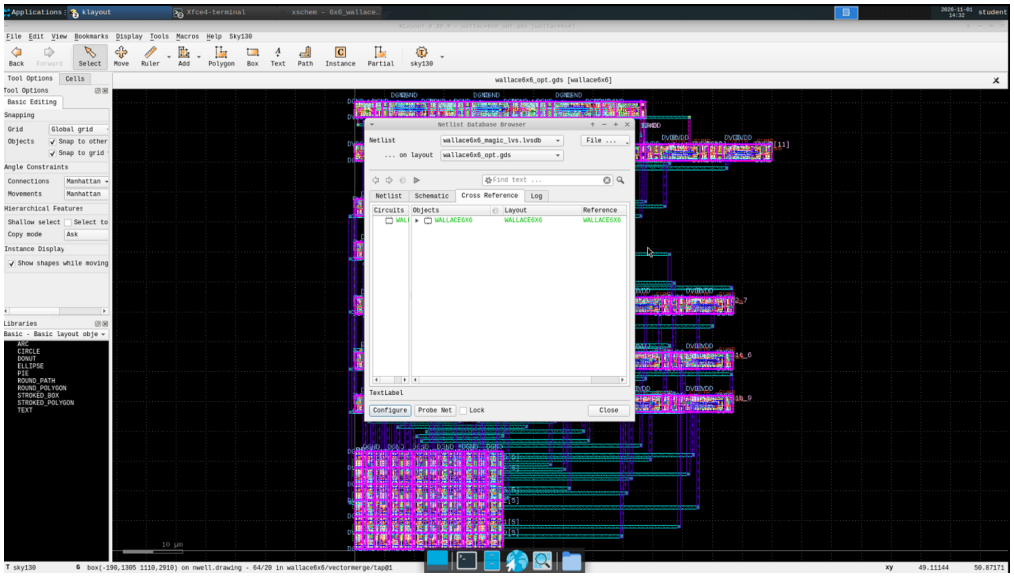


Figure 19: Netgen LVS result for `wallace6x6`.

9.3 Parasitic extraction

For post-layout simulation the layout was extracted with parasitics using `run_magic_extract_fixed.sh`:

1. `gds read` the final GDS, `readspice` to adopt the schematic port order, then reload and flatten the top cell `wallace6x6`;
2. `extract all`, `ext2sim`, and `extresist` (tolerance 10) for wire resistance;
3. `ext2spice` with `cthresh 0` (keep all capacitances) and `extresist on`, written to `wallace6x6_extracted.spice`.

The extracted netlist contains the same 1,268 transistors plus 9,095 parasitic capacitors and 9,916 parasitic resistors, and is used as the device under test in the post-layout testbench.

10 Critical Path Analysis

The critical path was found in two phases. First, the textbook (analytical) method was applied. Its prediction was then checked in SPICE and did not match the simulated circuit, so the final critical path was determined by transistor-level simulation instead. Both phases are described below, because the mismatch itself explains why the final path is what it is.

10.1 Phase 1: textbook (analytical) estimate

Delay model. Following the standard treatment of tree multipliers (Rabaey, *Digital Integrated Circuits*, multipliers chapter; Weste & Harris, datapath chapter), the delay of a Wallace multiplier is split into three parts:

$$t_{mult} \approx t_{AND} + N_{stages} \cdot t_{FA} + t_{merge},$$

with $N_{stages} = 3$ here. For a ripple-type merge of n bits,

$$t_{merge} \approx (n-1)\, t_{carry} + t_{sum},$$

and for the carry-bypass merge, a carry entering at the bottom of the merge can skip any 3-bit block whose bits all propagate.

Step 1: longest structural (topological) path.

- The sum output of a full adder (two XOR levels) is slower than its carry output (one majority gate), and the sum stays in the same column at the next stage. The slowest route through the tree is therefore *sum* $\rightarrow$ *sum* $\rightarrow$ *sum* in one column.
- The two final rows start at column 4, so the longest carry chain enters the merge at column 4 and ripples to the top (column 11).

This gives the topological worst case

$$t_{topo} = t_{AND} + 3\, t_{sum} + 7\, t_{carry} + t_{sum} \quad \text{(ending at } P_{11}\text{)}.$$

Step 2: sensitisation (false-path removal). A path only counts if some input change can actually travel along it. In the merge a carry waits for its carry-in only in propagate mode ($x_k \neq y_k$); in generate (1,1) or kill (0,0) mode the path is blocked. Since X and Y are produced by the tree and are not free inputs, not every pattern is possible. Checking all $64 \times 64 \times 12$ single-bit input transitions on a gate-level model showed that **no input can ripple a carry from column 4 to** P_{11}; the topological path is a *false path*. Two structural facts limit the real paths:

- block 1's carry-in is tied to 0, so any change in its carry-out starts inside block 1 and leaves through the rippled (c_2) input of its multiplexer;
- a carry cannot ripple through all three bits of block 2, because if all three bits propagate the multiplexer takes the bypass. A long ripple in block 2 therefore ends in one of its sum outputs, P_7–P_9.

The longest sensitisable path is therefore

$$t_{crit} = t_{AND} + 3\, t_{sum} + 5\, t_{carry} \text{ (columns 4–8)} + t_{sum} \text{ (column 9)} \quad \text{(ending at } P_9\text{)}.$$

Step 3: unit-delay estimate and predicted input. With textbook-style unit delays $t_{AND} = 1$, $t_{sum} = 2$, $t_{carry} = 1.5$ (equal for rising and falling outputs):

$$t_{topo} = 1 + 6 + 10.5 + 2 = 19.5 \text{ units (false)}, \qquad t_{crit} = 1 + 6 + 7.5 + 2 = 16.5 \text{ units}.$$

The input that sensitises this longest chain (column 4 generates, columns 5–8 propagate) is $B = 27$ held with A: $19 \rightarrow 3$ (a_4 falling). It makes six merge carries switch and was the predicted worst-case input.

10.2 Phase 2: checking the prediction in SPICE

The predicted input was simulated in ngspice (SKY130 `tt`, 1.8 V, 5 fF loads), measuring the time from the input edge (50 %) until the whole product settles.

	Textbook prediction	SPICE result
Worst-case input	A: $19 \to 3$, $B = 27$	$A = 35$, B: $15 \to 14$
Last output to settle	P_9	P_7 for the predicted input; P_9 for the true worst case
Delay of predicted input	longest (16.5 units)	2.28 ns, *not* the slowest
Delay of true worst case	–	2.74 ns (0.46 ns, 20 % slower)
Rise vs. fall	equal	falling 2.25 ns vs. rising 1.30 ns on the same path

Table 7: Textbook prediction compared with the SPICE simulations.

Why the textbook method failed for this circuit.

1. **Rise and fall delays are very different.** The library full adder is a mirror adder whose internal nodes are pulled up through series, minimum-size PMOS transistors. A *falling* carry is therefore much slower than a rising one: on the same path, 2.25 ns falling versus 1.30 ns rising. The textbook model uses one delay per gate, so it cannot tell which transition is slow.

2. **The longest chain is not the slowest chain.** The predicted input switches six merge carries, but some of them rise (fast) and the last output to settle is P_7. Other inputs switch fewer carries but all in the falling direction, and are slower overall.

3. **Delay depends on the data.** How long a gate takes depends on which of its inputs switch, on glitches arriving from other paths, and on the load. A single number per gate ignores this.

4. **Calibrated gate models still ranked poorly.** Separate rise and fall delays for each cell were fitted to SPICE results and the gate-level model was rerun. Its ranking of inputs still agreed poorly with SPICE (rank correlation only 0.1–0.4). One of the slowest inputs found later changes no merge carry at all, which no carry-chain-based estimate would flag.

Conclusion of Phase 2: the textbook analysis is useful to understand the *structure* (which paths are false, where long paths must end), but it cannot pick the worst-case input for this cell library. The critical path was therefore determined by transistor-level simulation.

10.3 Phase 3: simulation-based search

All runs use ngspice, SKY130 `tt`, 1.8 V, 5 fF on every output. Each candidate is a single-bit transition, placed in its own time window of one long transient run. Its delay is the time from the input's 50 % point until the decoded product equals its final value for good (every output past 50 %).

1. **Shortlist (48 runs).** All 49,152 single-bit transitions were ranked with the gate-level model under 12 different rise/fall delay assumptions. From the top of every ranking, one representative per distinct path was kept and simulated.

2. **Local search.** Every transition differing from the 8 slowest by one held input bit, and every other switching bit from the two slowest starting states, was simulated. None was slower than the leader, so the leader is a local maximum.

3. **Random baseline (20 runs).** Random single-bit transitions gave a median of 1.59 ns and a maximum of 2.11 ns, well below the leaders, which confirms the search was looking in the right region.

In total **176 transitions** were simulated, all with correct products.

Rank	Input transition	Switching bit	Settling (pre-layout)	Last output
1	$A = 35$, B: $15 \to 14$	b_0 falls	**2.74 ns**	P_9
2	A: $51 \to 35$, $B = 14$	a_4 falls	2.70 ns	P_9
3	$A = 45$, B: $44 \to 45$	b_0 rises	2.68 ns	P_9
4	$A = 45$, B: $41 \to 45$	b_2 rises	2.66 ns	P_9
⋮				
–	A: $19 \to 3$, $B = 27$ (textbook prediction)	a_4 falls	2.28 ns	P_7
–	20 random transitions	–	median 1.59, max 2.11 ns	–

Table 8: Slowest transitions found by the SPICE search (pre-layout).

The four slowest transitions all end at P_9 and lie within 0.1 ns of each other: they are different ways of sensitising the same falling carry chain through both blocks.

10.4 Critical path and activating input

Held input	$A = 35$ (100011) held
Switching input	B: 15 (001111) $\to$ 14 (001110) (b_0 falling)
Product	$525 \to 490$
Last output to settle	P_9
Critical delay	2.74 ns (pre-layout), 3.47 ns (post-layout)

Table 9: Input transition that activates the critical path.

The path, net by net. With $A = 35$ ($a_5 = a_1 = a_0 = 1$), the bit b_0 only affects a_0b_0, a_1b_0 and a_5b_0. The first two are in columns 0 and 1 and settle quickly. The third starts the long path:

$$
\begin{aligned}
& b_0{\downarrow} \to\ a_5b_0\ (\texttt{pp0[5]},\ \text{AND2, column 5}) \ \to\ \text{Stage 1 FA } \texttt{x5 : s1t_5} \\
& \to\ \text{Stage 2 FA } \texttt{x4 : s2_5} \ \to\ \text{Stage 3 FA } \texttt{x2 : s3_5} = X_5 \\
& \to\ \underbrace{\text{carry col. 5} \to \text{carry col. 6 } (c_2)}_{\text{block 1 ripple}} \ \to\ \text{MUX}_1\ (C_{out,1}) \\
& \to\ \underbrace{\text{carry col. 7} \to \text{carry col. 8}}_{\text{block 2 ripple}} \ \to\ \text{sum col. 9} \ \to\ P_9{\downarrow}\,.
\end{aligned}
$$

It contains 1 AND gate, 3 adder *sum* delays in the tree, 4 adder *carry* delays (all falling), 1 multiplexer and 1 final sum.

Node	Time after b_0 edge	Meaning
`s3_5` (X_5) falls	≈1.2 ns	end of the adder tree (AND + 3 sums)
block-1 C_{out} falls	≈1.95 ns	two block-1 carries + mux
P_7 settles	2.27 ns	carry enters block 2
P_8 settles	2.50 ns	
P_9 settles	**2.74 ns**	last output; product final

Table 10: Arrival times along the critical path. P_7, P_8, P_9 settle about 0.23 ns apart: the falling carry rippling one column at a time.

Timeline along the path (pre-layout, from the internal-node waveforms in Fig. 24). About 44 % of the delay is spent in the tree and 56 % in the merge adder. Block 2's carry-out never changes, so nothing reaches P_{10} or P_{11}.

Why this input activates the path. Table 11 gives the mode of each merge column before and after the transition, computed from the netlist.

Merge column	4	5	6	7	8	9	10	11
$B = 15$ (before)	kill	**generate**	propagate	propagate	propagate	kill	kill	kill
$B = 14$ (after)	kill	**propagate**	propagate	propagate	propagate	kill	kill	kill

Table 11: Generate/propagate/kill state of the vector-merge columns for $A = 35$.

- Before the edge, column 5 *generates* a carry and columns 6, 7, 8 *propagate* it, so all carries from column 5 up to column 9 are 1 and $P_9 = 1$.
- When b_0 falls, X_5 falls, column 5 changes from generate to propagate, and since column 4 kills, its carry-out becomes 0. This falling carry must ripple through columns 6, 7 and 8 before P_9 can fall.
- **Neither bypass helps.** Block 1 (columns 4–6) contains a kill column, so $\overline{BP}_1 = 1$ and its mux passes the rippled carry c_2. Block 2 (columns 7–9) also has a kill column (9), so the carry must ripple through columns 7 and 8.
- **All four carries fall**, the slow direction of the mirror adder. This is exactly the effect the textbook model missed.

While the carry ripples, the decoded product passes through wrong intermediate values (525 → … → 1000) before settling at 490, which is visible in the product waveform (Fig. 22).

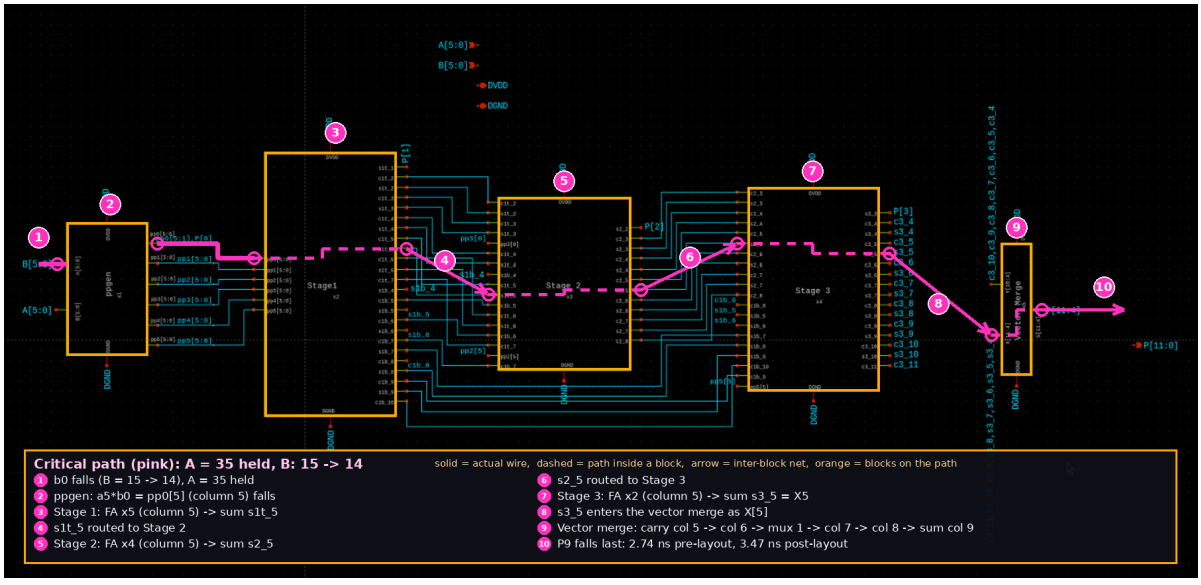


Figure 20: Critical path traced on the schematic.

11 Critical-Path Delay: Pre-layout vs Post-layout

11.1 Testbench

Both testbenches apply the same stimulus and loads:

- $A = 35$ (`100011`) on DC sources; $b_1 = b_2 = b_3 = 1$, $b_4 = b_5 = 0$ on DC sources;
- b_0 driven by `PWL(0 0 2n 0 2.05n 1.8 12n 1.8 12.05n 0)`: a rising edge at 2 ns (B 14→15, reference) and the critical falling edge at 12 ns (B 15→14), both with 50 ps transition time;
- 5 fF load on every output `p0`–`p11`;
- SKY130 `tt` corner, $V_{DD} = 1.8$ V, transient `tran 1p 22n`.

The product is decoded inside ngspice as `prod` $= \sum_k 2^k \, [\, v(p_k) > 0.9 \,]$. The settling time is the last instant after the b_0 edge (50 %) at which `prod` still differs from its final value; this includes every output and every glitch, not only P_9. The functional check measures `prod` at 1.9, 11.9 and 21.9 ns (expected 490, 525, 490).

- **Pre-layout:** `Wallace_sim` uses the schematic netlist of `wallace6x6`.
- **Post-layout:** `Wallace_sim_post.sch` includes `wallace6x6_extracted.spice` (1,268 MOSFETs with RC parasitics) as `Xdut`, with exactly the same sources and loads.

11.2 Results

Measurement	Pre-layout	Post-layout (RC)	Increase
Critical delay: $b_0\downarrow\rightarrow$ product settled	**2.74 ns**	**3.47 ns**	**+0.73 ns (+26.7 %)**
$b_0\downarrow\rightarrow P_9$ (last output)	2.74 ns	3.47 ns	+0.73 ns
$b_0\downarrow\rightarrow P_8$	2.50 ns	3.21 ns	+0.71 ns
$b_0\downarrow\rightarrow P_7$	2.27 ns	2.88 ns	+0.61 ns
Carry step per column in block 2	≈0.23 ns	≈0.30 ns	+30 %
Opposite edge: $b_0\uparrow\rightarrow$ product settled	1.32 ns	1.88 ns	+0.56 ns (+42 %)
Functional check (490/525/490)	Pass	Pass	–

Table 12: Critical-path delay before and after layout (`tt`, 1.8 V, 5 fF per output).

11.3 Simulator output

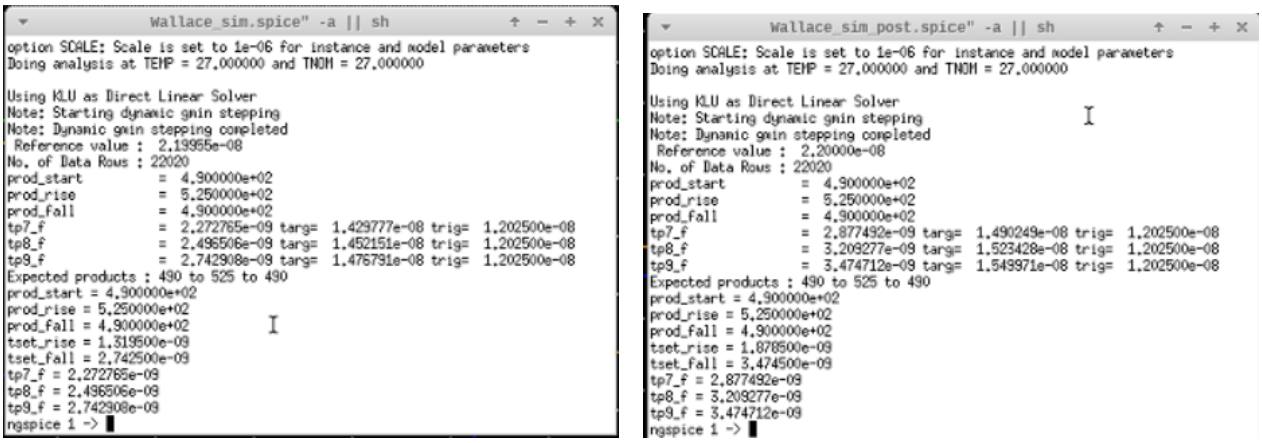


Figure 21: ngspice measurement output. `prod_start/rise/fall` confirm the products 490, 525, 490; `tset_fall` is the critical delay (2.7425 ns and 3.4745 ns); `tp7_f`, `tp8_f`, `tp9_f` are the delays to P_7, P_8, P_9.

11.4 Waveforms

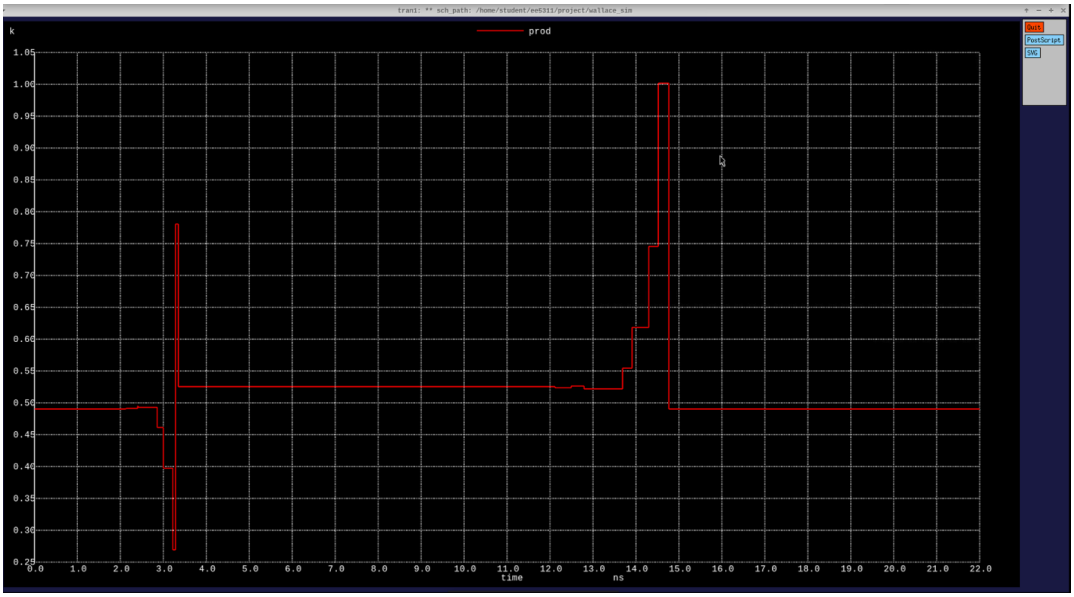


Figure 22: Pre-layout: decoded product `prod`. It is 490, rises to 525 when b_0 rises (2 ns) and returns to 490 when b_0 falls (12 ns). After the falling edge it passes through wrong intermediate values while the carry ripples, and settles 2.74 ns after the edge.

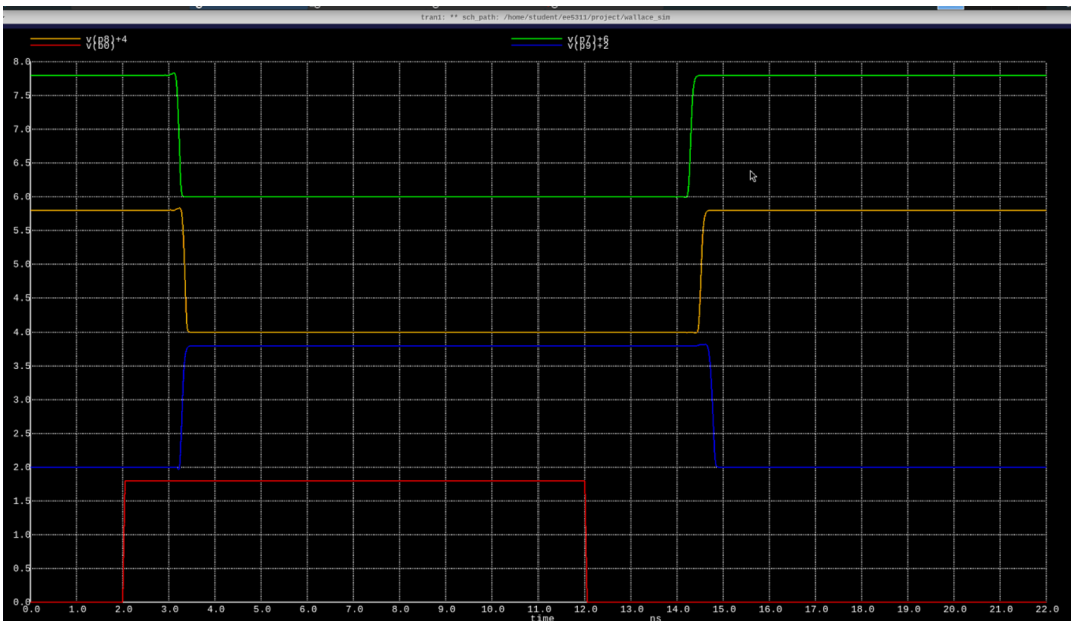


Figure 23: Pre-layout: b_0, P_9, P_8, P_7 (offset vertically). On the falling edge P_7, P_8 and P_9 switch one after another (2.27, 2.50, 2.74 ns), showing the falling carry rippling through block 2.

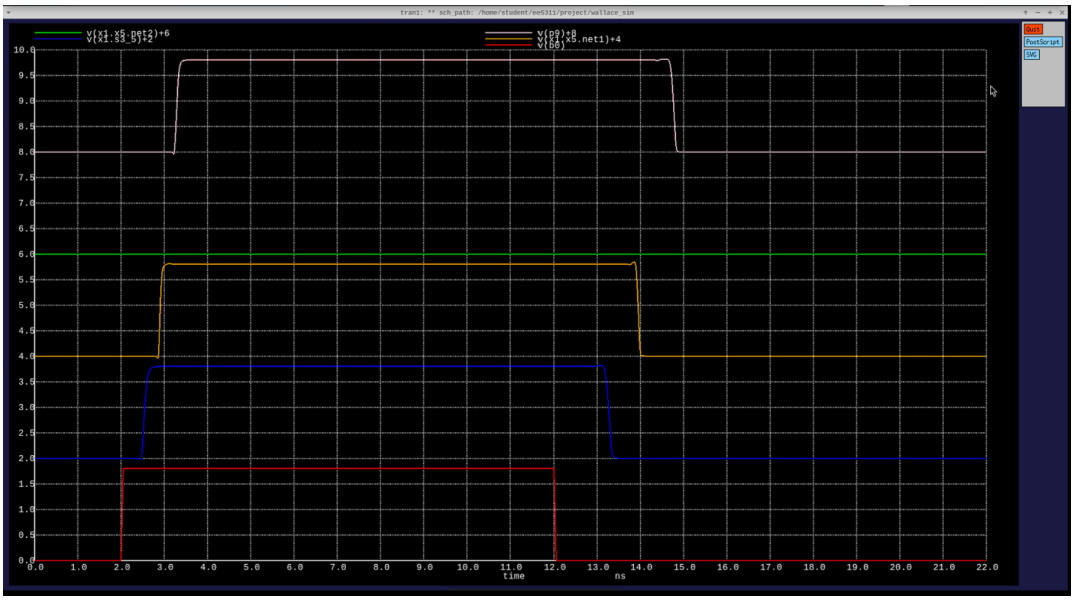


Figure 24: Pre-layout internal nodes along the critical path (offset vertically): b_0, the column-5 tree output `s3_5`, the bypass-block carry outputs inside the vector merge, and P_9. The signal moves from the tree output ($\approx$1.2 ns) through block 1 ($\approx$1.95 ns) to P_9 (2.74 ns).

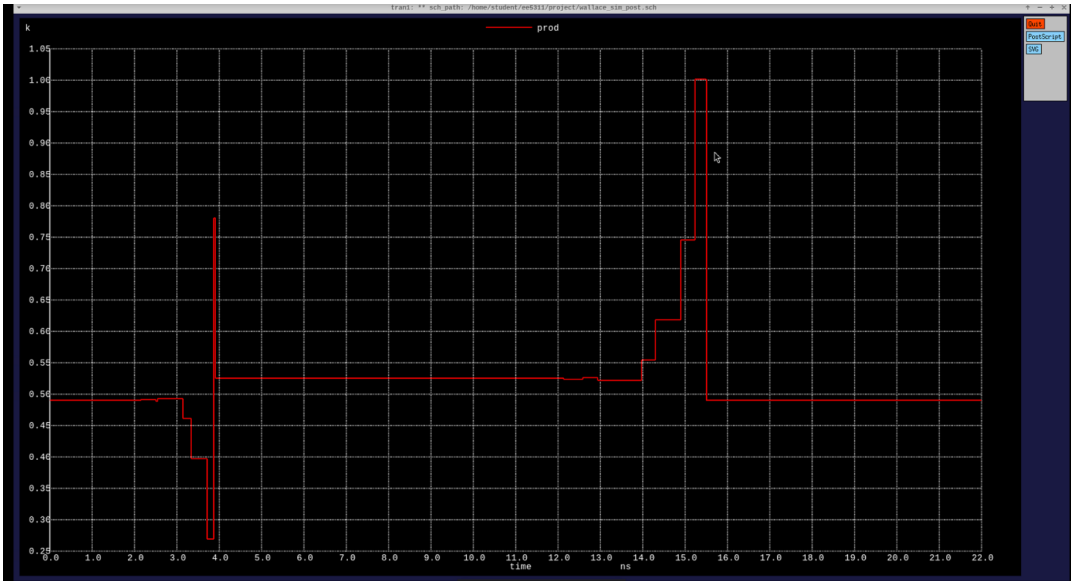


Figure 25: Post-layout (RC-extracted): decoded product. The same sequence 490, 525, 490 is obtained; after the falling edge the product settles 3.47 ns after the edge.

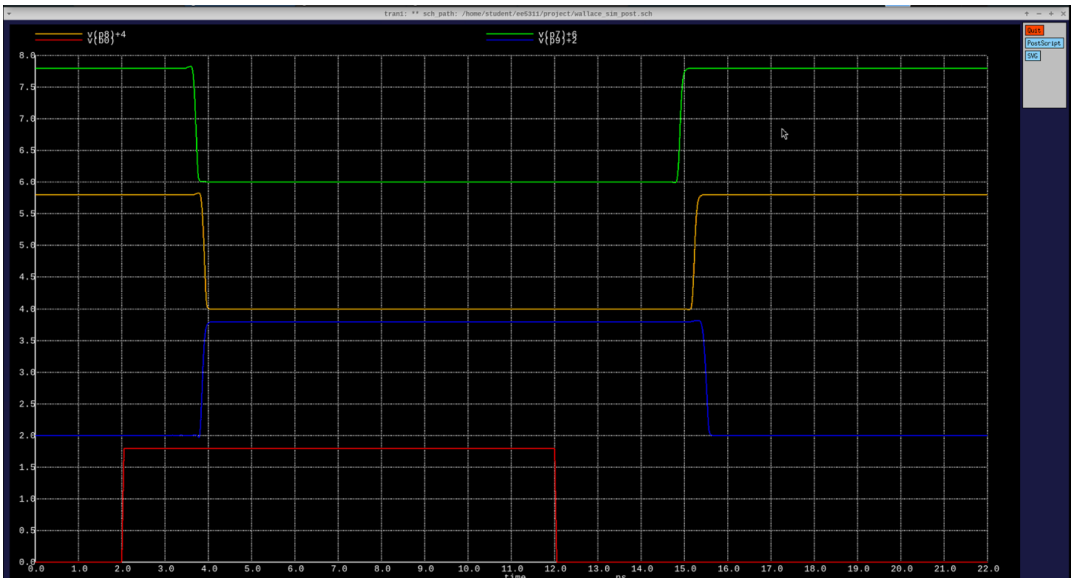


Figure 26: Post-layout: b_0, P_9, P_8, P_7. The ripple order is unchanged, but each output is later (2.88, 3.21, 3.47 ns) because of the wiring parasitics.

11.5 Discussion

- **The layout adds 0.73 ns (+26.7 %)** to the critical delay. The extracted netlist adds 9,095 wire capacitances and 9,916 wire resistances that the schematic does not have.
- **Every column of the ripple slows down by the same amount.** The carry step in block 2 grows from about 0.23 ns to 0.30 ns per column, and the increase grows along the chain (P_7 +0.61, P_8 +0.71, P_9 +0.73 ns). The added delay therefore accumulates along the carry chain rather than coming from one long wire.
- **The critical path is the same before and after layout**: P_9 is the last output to settle in both, and the functional result is correct in both. The layout changes how long the path takes, not which path is critical.
- **Where the delay is added.** The critical path crosses three blocks (Stage 1 → Stage 2 → Stage 3 → vector merge) through long top-level `met3`/`met4` routes, and its four carries are all falling, which is the weak pull-up direction of the mirror adder, so the extra wire load costs most there.
- **The rising edge degrades more in percent** (+42 %) but stays much faster in absolute terms (1.88 ns), so the falling edge remains critical after layout.
- The worst-case delay corresponds to a maximum operating rate of about $1/2.74\,\text{ns} \approx 365\,\text{MHz}$ before layout and $1/3.47\,\text{ns} \approx 288\,\text{MHz}$ after layout (combinational, `tt` corner, without register overhead).

11.6 Limitation

Simulating every possible input transition at transistor level ($\approx 1.7 \times 10^7$ pairs) is not feasible. The reported critical path is the worst case found by the structured search: it is slower than every one of the 176 simulated transitions and is a local maximum among single-bit transitions, and it agrees with the structural analysis (long falling ripple through both blocks, ending in a block-2 sum).

12 Conclusion

A 6×6 unsigned Wallace tree multiplier was designed in SKY130 from the supplied cell library. The design uses 36 AND gates for partial products, three reduction stages (17 full adders and 9 half adders, the half adders being full adders with one input grounded), and a vector merge made of two 3-bit carry-bypass blocks followed by two full adders. The design was verified exhaustively at gate level, its layout is DRC-clean with 0 violations and LVS-clean ("Circuits match uniquely"), and pre- and post-layout simulations give correct products. Sensitisation analysis showed that the structurally longest path to P_{11} is false, and a gate-level screening of all 49,152 single-bit transitions gave the textbook estimate, but its predicted worst-case input proved not to be the slowest in SPICE because the mirror adder's falling carries are much slower than its rising ones. A 176-transition SPICE search then identified the critical input: $A = 35$ held while b_0 falls (B: 15→14). It launches a falling carry from column 5 through block 1, its bypass multiplexer and block 2 to P_9. The critical delay is 2.74 ns before layout and 3.47 ns after layout, an increase of +0.73 ns (+26.7 %) caused by the extracted wiring parasitics; the critical path is the same in both cases.